

**ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



Thực tập 2

**PHÂN TÍCH HIỆU NĂNG CỦA
JAVASCRIPT VÀ WEBASSEMBLY TRONG XỬ LÝ
VIDEO THỜI GIAN THỰC TRÊN TRÌNH DUYỆT WEB**

Giáo viên hướng dẫn: PGS.TS Thoại Nam

Sinh viên thực hiện:

Võ Hoàng Quốc 2470310

TP. HCM, 5/2026

**ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



Thực tập 2

**PHÂN TÍCH HIỆU NĂNG CỦA
JAVASCRIPT VÀ WEBASSEMBLY TRONG XỬ LÝ
VIDEO THỜI GIAN THỰC TRÊN TRÌNH DUYỆT WEB**

Giáo viên hướng dẫn: PGS.TS Thoại Nam

Sinh viên thực hiện:

Võ Hoàng Quốc 2470310

TP. HCM, 5/2026

Lời cảm ơn

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc nhất đến **PGS. TS Thoại Nam**, người thầy đã trực tiếp hướng dẫn, tận tình chỉ bảo và truyền đạt những kinh nghiệm quý báu cho em trong suốt quá trình thực hiện đề tài. Sự định hướng chuyên môn khắt khe nhưng đầy tâm huyết của Thầy chính là chìa khóa giúp em giải quyết được những bài toán khó về hiệu năng giữa JavaScript và WebAssembly trong xử lý video thời gian thực.

Em cũng xin gửi lời cảm ơn chân thành đến quý Thầy, Cô khoa **Khoa học và Kỹ thuật Máy tính** cùng tập thể giảng viên **Trường Đại học Bách khoa – ĐHQG TP.HCM**. Trong suốt những năm học tập tại trường, em đã nhận được sự giáo dục tận tâm, những kiến thức nền tảng vững chắc và môi trường học tập tuyệt vời để có thể tự tin thực hiện công trình nghiên cứu này.

Cuối cùng, em xin kính chúc **PGS. TS Thoại Nam** cùng quý Thầy, Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp cao quý của mình.

Em xin chân thành cảm ơn!

Tóm tắt

Sự chuyển dịch của các tác vụ tính toán hiệu năng cao từ mô hình tập trung tại máy chủ sang thực thi trực tiếp tại máy khách đang trở thành một xu thế tất yếu nhằm tối ưu hóa độ trễ và tăng cường tính riêng tư của dữ liệu. Tuy nhiên, bản chất kiểu động và cơ chế quản lý bộ nhớ tự động của JavaScript vẫn là rào cản lớn đối với các tác vụ xử lý đa phương tiện thời gian thực độ phân giải cao. Báo cáo **Phân tích hiệu năng của JavaScript và WebAssembly trong xử lý video thời gian thực trên trình duyệt web** tập trung vào bài toán xử lý video 4K – một đại diện cho lớp bài toán xử lý pixel cường độ cao.

Nghiên cứu áp dụng phương pháp thực nghiệm hệ thống, xây dựng một framework benchmark chuẩn hóa trên trình duyệt Google Chrome để đo lường định lượng hiệu năng của pipeline phát hiện chuyển động. Hệ thống được thiết kế theo mô hình 4 kịch bản tối ưu hóa lũy tiến: (1) Wasm Scalar, (2) Wasm SIMD 128-bit, (3) Wasm Multi-threading và (4) Kết hợp Hybrid SIMD + Multi-threading. Các chỉ số đo lường cốt lõi bao gồm Latency, FPS, P99 và Độ lệch chuẩn để đánh giá toàn diện cả tốc độ thực thi lẫn tính ổn định của hệ thống.

Kết quả thực nghiệm cho thấy WebAssembly Scalar cung cấp hiệu năng cao hơn nhờ mã nhị phân kiểu tĩnh, quá trình biên dịch tối ưu trước khi thực thi và mô hình bộ nhớ ổn định hơn, qua đó giảm ảnh hưởng của các rủi ro warm-up hoặc deoptimization thường gặp trong JavaScript JIT. Đặc biệt, sự kết hợp giữa SIMD và Multi-threading giúp độ trễ xử lý lỗi giảm xuống 11.13 ms/khung hình, thấp hơn ngân sách 16.67 ms tương ứng với mục tiêu 60 FPS. Tuy nhiên, thông lượng đầu cuối đo được vẫn đạt 27.9 FPS do còn chịu chi phí sao chép dữ liệu, điều phối qua JavaScript-Wasm bridge và hiển thị lên Canvas. Phân tích chuyên sâu cũng chỉ ra rằng trong khi JavaScript gặp hiện tượng giạt khung hình nặng nề do áp lực bộ nhớ (~33MB/frame), WebAssembly duy trì một hiệu năng xác định với độ lệch chuẩn cực thấp (~1ms) nhờ khả năng quản lý bộ nhớ thủ công.

Nghiên cứu cũng xác định điểm bão hòa hiệu năng khi bài toán chuyển dịch từ giới hạn tính toán sang giới hạn băng thông bộ nhớ ở các cấu hình tối ưu cao nhất. Những kết luận thực chứng này cung cấp các khuyến nghị kỹ thuật quan trọng cho các nhà phát triển trong việc lựa chọn chiến lược tối ưu hóa cho các ứng dụng Web Media thế hệ mới.

Từ khóa: *WebAssembly, JavaScript, Hiệu năng, Phát hiện chuyển động, SIMD, Multi-threading, Xử lý video thời gian thực.*

Mục lục

Lời cảm ơn	2
Tóm tắt	3
Mục lục	4
Danh mục hình ảnh	6
Danh mục bảng biểu	7
Danh mục từ viết tắt	8
CHƯƠNG 1: GIỚI THIỆU	10
1.1. Bối cảnh và Thách thức trong phát triển ứng dụng Web hiện đại	10
1.2. Công trình liên quan và Khoảng trống nghiên cứu	12
1.3. Động lực, Mục tiêu và Câu hỏi nghiên cứu	15
1.4. Phạm vi và Phương pháp nghiên cứu	16
1.5. Đóng góp và Cấu trúc báo cáo	18
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	20
2.1. Tổng quan về tính toán hiệu năng cao trên trình duyệt	20
2.2. Các kỹ thuật tối ưu hóa song song trên kiến trúc CPU hiện đại	22
2.3. Thuật toán xử lý video thời gian thực	24
2.4. Khung đánh giá hiệu năng	26
2.5. Tổng quan các công trình nghiên cứu liên quan	27
CHƯƠNG 3: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG	29
3.1 Mục tiêu và Nguyên tắc thiết kế	29
3.2 Kiến trúc hệ thống tổng thể	30
3.3 Thiết kế tầng lưu trữ và Giao tiếp	32
3.4 Cài đặt các kịch bản thực nghiệm	34
3.5 Môi trường thực nghiệm	36
3.6 Phương pháp Benchmark và Thu thập dữ liệu	37
CHƯƠNG 4: KẾT QUẢ VÀ PHÂN TÍCH	39
4.1 Giới thiệu chương	39
4.2 Kết quả thực nghiệm các kịch bản đơn lẻ	39
4.4. Phân tích chuyên sâu và Thảo luận	45

4.5 Tổng kết chương	46
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
5.1 Kết luận	47
5.2 Đóng góp của báo cáo	49
5.3 Hạn chế và Hướng phát triển tương lai	51
Tài liệu tham khảo	54

Danh mục hình ảnh

Hình 1.1	Mô hình chuyển dịch từ tính toán phía Máy chủ sang Máy khách	10
Hình 1.2	Minh họa hiện tượng Jitter do Garbage Collection gây rớt khung hình	11
Hình 2.1	Hệ sinh thái WebAssembly	20
Hình 2.2	Pipeline thực thi của JavaScript JIT vs. WebAssembly	21
Hình 2.3	Mô hình SISD vs Mô hình SIMD	22
Hình 2.4	Kiến trúc Đa luồng với SharedArrayBuffer	23
Hình 2.5	Pipeline Thuật toán Phát hiện Chuyển động	24
Hình 2.6	Tối ưu hóa Box blur bằng Separable Filter	25
Hình 3.1	Sơ đồ khối kiến trúc hệ thống tổng thể	31
Hình 3.2	Luồng xử lý dữ liệu	31
Hình 3.3	Cơ chế sao chép dữ liệu qua Cầu nối	33
Hình 3.4	Chiến lược tối ưu hóa tích lũy	34
Hình 3.5	Hai giai đoạn của benchmark	38
Hình 4.1	Performance panel của chrome trong kịch bản số 4	44

Danh mục bảng biểu

Bảng 1.1	Bảng các công trình nghiên cứu liên quan	14
Bảng 2.1	Bảng so sánh quy trình thực thi giữa JavaScript và WebAssembly:	22
Bảng 4.1	Kịch bản 1: WebAssembly scalar	40
Bảng 4.2	Kịch bản 2: WebAssembly SIMD	41
Bảng 4.3	Kịch bản 3: WebAssembly Multi-threaded	42
Bảng 4.4	Kịch bản 4: WebAssembly SIMD + Multi-threaded	43
Bảng 5.1	Tổng hợp câu trả lời cho các câu hỏi nghiên cứu	49

Danh mục từ viết tắt

STT	Chữ viết tắt	Chữ đầy đủ
1	Wasm	WebAssembly
2	SIMD	Single Instruction Multiple Data
3	FPS	Frames Per Second
4	P99	99th Percentile
5	JIT	Just-in-Time
6	SISD	Single Instruction Single Data
7	AOT	Ahead-of-Time
8	AI/ML	Artificial Intelligence / Machine Learning
9	CPU	Central Processing Unit
10	V8	V8 JavaScript Engine
11	C++	C Plus Plus
12	SPEC	Standard Performance Evaluation Corporation
13	UHD	Ultra High Definition
14	DOM	Document Object Model
15	I/O	Input/Output
16	RQ	Research Question
17	GPU	Graphics Processing Unit
18	WebGPU	Web Graphics Processing Unit
19	W3C	World Wide Web Consortium
20	VM	Virtual Machine

21	GC	Garbage Collection
22	SAB	SharedArrayBuffer
23	POSIX	Portable Operating System Interface
24	RAM	Random Access Memory
25	COOP	Cross-Origin-Opener-Policy
26	COEP	Cross-Origin-Embedder-Policy
27	RGBA	Red Green Blue Alpha
28	ITU-R	International Telecommunication Union - Radiocommunication Sector
29	API	Application Programming Interface
30	HTTP	Hypertext Transfer Protocol
31	JS	JavaScript
32	SPA	Single Page Application
33	HTML	HyperText Markup Language
34	LLVM	Low Level Virtual Machine
35	ALU	Arithmetic Logic Unit
36	FPU	Floating Point Unit
37	MP4	MPEG-4 Part 14
38	OS	Operating System
39	MT	Multi-threaded
40	KB	Kịch bản

CHƯƠNG 1: GIỚI THIỆU

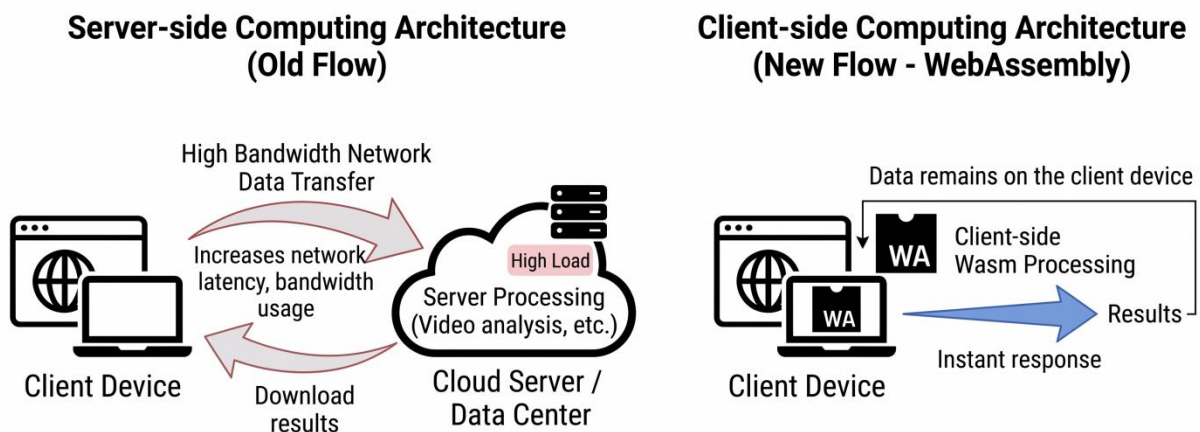
1.1. Bối cảnh và Thách thức trong phát triển ứng dụng Web hiện đại

Sự phát triển mạnh mẽ của các tiêu chuẩn web và năng lực phần cứng trong thập kỷ qua đã biến trình duyệt từ một công cụ hiển thị tài liệu tĩnh thành một nền tảng thực thi ứng dụng toàn diện. Tuy nhiên, sự kỳ vọng ngày càng cao của người dùng đối với các trải nghiệm tương tác phức tạp đã tạo ra một nghịch lý lớn: *nhu cầu về các ứng dụng tính toán hiệu năng cao đang vượt xa khả năng đáp ứng của các kiến trúc thực thi dựa trên kịch bản truyền thống.*

1.1.1. Sự chuyển dịch sang tính toán hiệu năng cao tại máy khách

Trong mô hình kiến trúc truyền thống, các tác vụ tính toán nặng thường được ủy thác cho các máy chủ có cấu hình mạnh mẽ. Tuy nhiên, xu hướng hiện nay đang chứng kiến sự chuyển dịch mạnh mẽ của các công việc xử lý đa phương tiện, trí tuệ nhân tạo (AI/ML), và mô phỏng 3D trực tiếp về phía máy khách (Client-side). Sự chuyển dịch này mang lại hai lợi ích cốt lõi:

1. **Tối ưu hóa độ trễ và băng thông mạng:** Việc xử lý dữ liệu tại chỗ loại bỏ nhu cầu gửi các luồng dữ liệu khổng lồ lên máy chủ, đảm bảo tính phản hồi tức thời cho các ứng dụng thời gian thực.
2. **Bảo mật và tính riêng tư:** Dữ liệu nhạy cảm của người dùng (như hình ảnh, video từ camera) không cần phải rời khỏi thiết bị cá nhân, đáp ứng các tiêu chuẩn khắt khe về bảo vệ quyền riêng tư hiện đại.



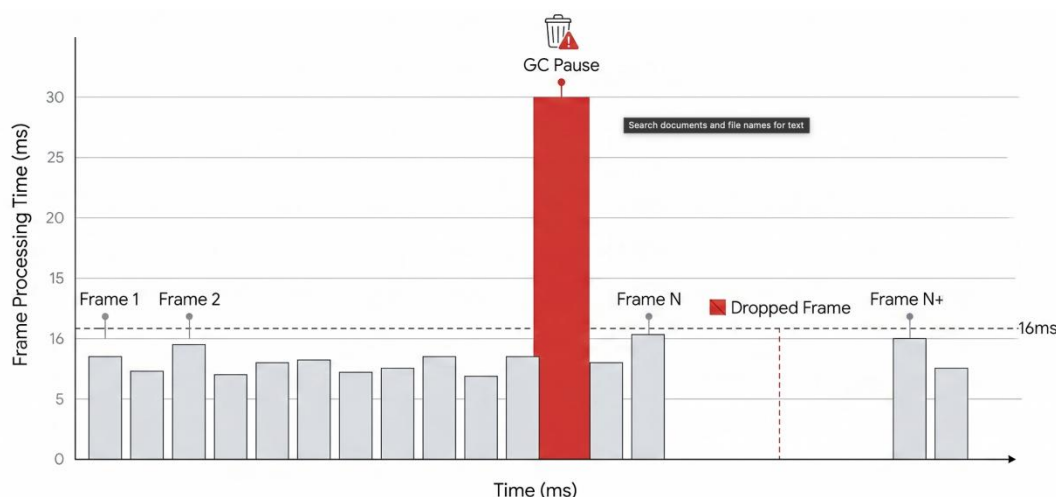
Hình 1.1 Mô hình chuyển dịch từ tính toán phía Máy chủ sang Máy khách

Mặc dù mang lại nhiều lợi thế, nhưng việc thực thi các luồng dữ liệu lớn — điển hình là video độ phân giải cao như 4K — trên trình duyệt đặt ra yêu cầu khắt nghiệt về năng lực xử lý của CPU và băng thông bộ nhớ. Điều này trở thành một "bài toán sống còn" đối với các ứng dụng Web Media hiện nay.

1.1.2. Các rào cản về hiệu năng của JavaScript

Mặc dù JavaScript là ngôn ngữ thống trị nền tảng Web, bản chất của nó lại chứa đựng những hạn chế đối với các tác vụ tính toán chuyên sâu.

- **Tính thiếu ổn định của cơ chế Just-In-Time (JIT):** JavaScript là ngôn ngữ kiểu động, phụ thuộc hoàn toàn vào các bộ tối ưu hóa JIT của trình duyệt (như V8 hay SpiderMonkey). Quá trình *warm-up* của JIT và các rủi ro từ việc *deoptimization* dẫn đến sự thiếu ổn định trong thời gian thực thi.
- **Vấn đề Jitter và Cơ chế quản lý bộ nhớ tự động:** Cơ chế quản lý bộ nhớ tự động (Garbage Collection) của JavaScript thường xuyên gây ra các khoảng dừng thực thi không mong muốn. Trong xử lý video, các khoảng dừng này biểu hiện qua hiện tượng rớt khung hình và jitter, làm giảm nghiêm trọng trải nghiệm người dùng.
- **Rào cản khai thác phần cứng:** JavaScript truyền thống gặp khó khăn trong việc khai thác trực tiếp các kiến trúc phần cứng hiện đại như tập lệnh SIMD (Single Instruction, Multiple Data) để song song hóa mức dữ liệu, hoặc xử lý đa luồng (multi-threaded) thực sự do bản chất đơn luồng (single-threaded) của Event Loop.



Hình 1.2 Minh họa hiện tượng Jitter do Garbage Collection gây rớt khung hình

1.1.3. WebAssembly và hứa hẹn về hiệu năng tiệm cận mã gốc

Sự ra đời của WebAssembly (Wasm) đánh dấu một bước ngoặt quan trọng, cung cấp một tiêu chuẩn mã nhị phân được thiết kế để hỗ trợ cho JavaScript. Wasm cho phép các nhà phát triển biên dịch mã nguồn từ các ngôn ngữ hệ thống như C, C++ hoặc Rust thành một định dạng có thể thực thi trên trình duyệt với tốc độ gần như mã gốc.

Với các ưu thế về kiểu dữ liệu tĩnh, mô hình bộ nhớ tuyến tính và khả năng tương thích cao với SIMD cũng như đa luồng, Wasm hứa hẹn sẽ giải phóng sức mạnh tính toán của trình duyệt khỏi các rào cản của JavaScript. Tuy nhiên, một vấn đề khoa học quan trọng được đặt ra là: ***Trong một pipeline xử lý video thực tế với khối lượng dữ liệu 4K, liệu sự cải thiện của Wasm có thực sự mang lại speedup tuyến tính như lý thuyết, hay***

sẽ bị giới hạn bởi các overhead của trình duyệt? Đề tài nghiên cứu này sẽ tập trung phân tích định lượng sự cải thiện hiệu năng thực tế của Wasm trong các kịch bản tối ưu hóa khác nhau so với JavaScript.

1.2. Công trình liên quan và Khoảng trống nghiên cứu

Việc đánh giá hiệu năng giữa Wasm và JavaScript đã trở thành một chủ đề thu hút sự quan tâm lớn của cộng đồng nghiên cứu kể từ khi Wasm được giới thiệu như một chuẩn nhị phân nhằm giúp các ứng dụng web đạt tốc độ thực thi tiệm cận các ứng dụng gốc. Các nghiên cứu hiện nay tập trung vào ba hướng chính:

1. Đánh giá hiệu năng tổng quát qua các bài thử nghiệm vi mô.
2. Ứng dụng Wasm trong các tác vụ chuyên biệt.
3. Khả năng khai thác các tính năng phần cứng nâng cao.

1.2.1. Đánh giá hiệu năng WebAssembly trong các tác vụ tính toán chuyên sâu

Phần lớn các nghiên cứu ban đầu tập trung vào việc so sánh Wasm và JavaScript thông qua các tập lệnh thử nghiệm vi mô (micro-benchmarks). Haas và cộng sự [5] đã đặt nền móng cho việc khẳng định Wasm có khả năng mang lại hiệu năng cao hơn đáng kể so với JavaScript nhờ vào định dạng nhị phân kiểu tĩnh. Ciszewski và Myk [6] cũng như Vemuri [8] chỉ ra rằng Wasm chiếm ưu thế vượt trội trong các bài toán ràng buộc bởi CPU như thuật toán Sieve of Eratosthenes, sắp xếp mảng hay các tính toán đệ quy. Tuy nhiên, JavaScript nhờ cơ chế JIT vẫn giữ được sự cạnh tranh trong một số kịch bản tính toán số thực đơn giản [6].

Dù có nhiều hứa hẹn, các đánh giá thực nghiệm quy mô lớn đã chỉ ra những "mảng tối" về hiệu năng. Jangda và cộng sự [1] qua các bài thử nghiệm SPEC CPU đã phát hiện ra rằng mã Wasm thực tế chạy chậm hơn mã gốc trung bình từ 45% đến 55%. Yan và cộng sự [2] cũng nhấn mạnh rằng hiệu năng của Wasm không đồng nhất, phụ thuộc lớn vào trình biên dịch, trình duyệt thực thi và mô hình bộ nhớ được sử dụng. Tǎlu [4] bổ sung rằng các môi trường thực thi khác nhau có sự khác biệt đáng kể về chi phí overhead khi trao đổi dữ liệu giữa JavaScript và Wasm.

1.2.2. Các ứng dụng Thị giác máy tính trên nền tảng Web

Thị giác máy tính là lớp bài toán điển hình cho việc xử lý dữ liệu khối lượng lớn. Taheri và cộng sự [3] đã giới thiệu OpenCV.js, một bước tiến lớn đưa thư viện thị giác máy tính chuẩn mực lên web thông qua biên dịch sang JavaScript/Wasm. Nghiên cứu này cho thấy các ứng dụng như phát hiện khuôn mặt hay trích xuất đặc trưng có thể hoạt động trực tiếp tại máy khách, dù vẫn tồn tại khoảng cách hiệu năng so với ứng dụng gốc.

Xu hướng nghiên cứu gần đây đã chuyển dịch sang việc tối ưu hóa hiệu năng ở mức tinh vi hơn. Cụ thể, nghiên cứu của Oishi và cộng sự [7] cho thấy việc tự xây dựng mã Wasm tùy chỉnh giúp tận dụng tối đa sức mạnh của các phép toán song song (vector operations). Cách tiếp cận này mang lại tốc độ xử lý các bộ lọc hình ảnh vượt trội so với

việc sử dụng các thư viện đa năng sẵn có như OpenCV.js". Parab [9] cũng khẳng định rằng vai trò của Wasm trong frontend đang ngày càng mở rộng, đặc biệt là khả năng tận dụng SIMD và đa luồng để giải quyết các bài toán xử lý dữ liệu theo thời gian thực.

Bảng 1.1 Bảng các công trình nghiên cứu liên quan

STT	Tác giả	Tên bài báo	Đối tượng & PPNC	Kết quả và Đóng góp
1	Jangda và cộng sự (2019)	Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code	So sánh Wasm với Native Code trên bộ Spec CPU.	Chỉ ra Wasm chậm hơn Native khoảng 45% - 55% và các hạn chế về tối ưu hóa của trình duyệt.
2	Yan và cộng sự (2021)	Understanding the Performance of WebAssembly Applications	Phân tích hiệu năng Wasm trên đa nền tảng và đa trình duyệt.	Khẳng định hiệu năng Wasm phụ thuộc lớn vào trình duyệt; Wasm không phải lúc nào cũng nhanh hơn JavaScript.
3	Taheri và cộng sự (2018)	OpenCV.js: Computer Vision Processing for the Web	Đưa thư viện OpenCV lên Web thông qua Wasm.	Cung cấp bộ công cụ xử lý thị giác máy tính chuẩn trên trình duyệt nhưng chưa đi sâu vào tối ưu phần cứng.
4	Tǎlu (2025)	A Comparative Study of WebAssembly Runtimes	So sánh các môi trường thực thi của Wasm.	Đánh giá các chỉ số hiệu năng, bảo mật và thách thức khi tích hợp Wasm vào các hệ thống hiện đại.
5	Haas và cộng sự (2017)	Bringing the Web up to Speed with WebAssembly	Đề xuất kiến trúc, định dạng nhị phân và ngữ nghĩa hình thức cho Wasm.	Thiết lập tiêu chuẩn cho Wasm, chứng minh tính an toàn và khả năng thực thi tốc độ cao.
6	Ciszewski (2025)	Performance Comparison of Web Assembly and JavaScript	Đánh giá tổng quát tốc độ thực thi giữa Wasm và JavaScript.	Xác nhận ưu thế của Wasm trong các tác vụ tính toán nặng nhưng cũng chỉ ra các rào cản về tương tác dữ liệu.
7	Oishi và cộng sự (2021)	Performance Evaluation of Image Convolution with WebAssembly	Sử dụng SIMD tối ưu bộ lọc tích chập trong xử lý ảnh.	Chứng minh mã Wasm tự viết tối ưu hóa tầng thấp (SIMD) nhanh hơn các thư viện tổng quát như OpenCV.js.
8	Vemuri (2025)	WebAssembly for High-Performance Web Applications	Nghiên cứu về tốc độ thực thi và hiệu quả quản lý bộ nhớ.	Khẳng định Wasm là giải pháp thay thế/bổ trợ hoàn hảo cho JavaScript trong Game và Xử lý đa phương tiện.
9	Parab (2025)	WebAssembly's Expanding Role in Frontend Development	Lộ trình phát triển và vai trò của Wasm trong Frontend.	Chỉ ra các kỹ thuật tối ưu hóa để cải thiện UI.

1.2.3. Khoảng trống nghiên cứu

Mặc dù các nghiên cứu hiện tại đã cung cấp cái nhìn tổng quan về tiềm năng của Wasm, vẫn tồn tại những hạn chế cốt lõi mà đề tài này hướng tới giải quyết:

1. **Thiếu hụt thực nghiệm với các tác vụ nặng:** Phần lớn các nghiên cứu hiện nay (như [1], [6], [7]) vẫn thực hiện trên các bài toán nhỏ hoặc xử lý ảnh tĩnh. Chưa có nhiều nghiên cứu đánh giá pipeline xử lý video thời gian thực trên độ phân giải **4K UHD**, nơi mà áp lực lên băng thông bộ nhớ và CPU là lớn nhất.
2. **Thiếu đánh giá về hiệu quả tối ưu tích lũy:** Hầu hết các công trình thường so sánh đơn lẻ: hoặc Wasm vs JavaScript [8], hoặc SIMD vs Scalar [7]. Hiện tại rất hiếm nghiên cứu phân tích một cách hệ thống hiệu quả cộng hưởng của bốn kịch bản tối ưu hóa lũy tiến: **Scalar -> SIMD -> Multi-threaded -> SIMD + Multi-threaded** trong cùng một hệ thống thực nghiệm thực tế.
3. **Vấn đề ổn định trong xử lý luồng dữ liệu:** Các nghiên cứu đa số tập trung vào thời gian thực thi trung bình mà bỏ qua các chỉ số về độ ổn định như **P99** hay **Jitter** trong một pipeline video dài hạn – yếu tố quyết định trải nghiệm người dùng thực tế.

Đề tài này sẽ lấp đầy các khoảng trống trên bằng cách xây dựng một framework benchmark chuẩn hóa, thực hiện đánh giá định lượng với các kịch bản tích lũy của các kỹ thuật tối ưu hóa trên pipeline xử lý video 4K thực tế.

1.3. Động lực, Mục tiêu và Câu hỏi nghiên cứu

1.3.1. Động lực chọn đề tài

Động lực thúc đẩy nghiên cứu này xuất phát từ sự giao thoa giữa nhu cầu thực tế của người dùng và các rào cản kỹ thuật hiện hữu trên nền tảng Web, được phân tích qua ba tầng bài toán sau:

- **Bài toán tổng quát – Nâng cao trải nghiệm người dùng tại máy khách:** Trong kỷ nguyên số hóa, người dùng ngày càng ưu tiên các ứng dụng Web có tính tương tác cao và khả năng xử lý dữ liệu tức thời. Việc chuyển dịch các tác vụ tính toán nặng từ máy chủ về máy khách không chỉ giúp giảm chi phí hạ tầng mà còn là yếu tố then chốt để đảm bảo tính riêng tư của dữ liệu và giảm thiểu độ trễ mạng [4][9].
- **Bài toán hẹp – Tối ưu hóa xử lý video thời gian thực:** Trong các loại hình dữ liệu, video thời gian thực là lớp công việc "nhảy cào" nhất với hiệu năng. Khi độ phân giải chuẩn hóa chuyển dần sang 4K, khối lượng pixel cần xử lý mỗi giây tăng lên đột biến, đòi hỏi một cơ chế thực thi cực kỳ ổn định. JavaScript, với các hạn chế về JIT và cơ chế quản lý bộ nhớ tự động, thường xuyên gây ra hiện tượng rớt khung hình, không đáp ứng được yêu cầu khắt khe của các ứng dụng media chuyên nghiệp [1][8].

- **Bài toán cụ thể – Motion Detection làm đại diện:** Đề tài lựa chọn thuật toán phát hiện chuyển động (Motion Detection) không phải với mục tiêu cải tiến thuật toán, mà sử dụng nó như một kịch bản thử nghiệm đại diện cho lớp bài toán xử lý pixel lặp lại cường độ cao. Đây là một bài toán lý tưởng để đo lường giới hạn chịu tải của trình duyệt và đánh giá hiệu quả của việc can thiệp sâu vào kiến trúc phần cứng thông qua Wasm.

1.3.2. Mục tiêu nghiên cứu

Nghiên cứu được thực hiện nhằm đạt được các mục tiêu cụ thể sau:

1. **Xây dựng hệ thống Benchmark chuẩn hóa:** Thiết kế một framework đo lường hiệu năng chuyên biệt trên trình duyệt, có khả năng cô lập các yếu tố gây nhiễu (DOM, I/O) để thu thập dữ liệu chính xác về thời gian thực thi của thuật toán trên luồng video 4K.
2. **Đánh giá định lượng sức mạnh của 4 mức tối ưu hóa Wasm:** Thực hiện so sánh đối chiếu giữa JavaScript baseline và Wasm qua bốn kịch bản lũy tiến: Scalar, SIMD, Multi-threaded, và kết hợp SIMD + Multi-threaded.
3. **Đề xuất khuyến nghị kỹ thuật:** Dựa trên các dữ liệu thực nghiệm, phân tích các điểm đánh đổi để cung cấp hướng dẫn cho các nhà phát triển trong việc lựa chọn cấu hình tối ưu khi xây dựng các ứng dụng Web Media hiệu năng cao.

1.3.3. Câu hỏi nghiên cứu

Đề định hướng cho các hoạt động thực nghiệm, đề tài tập trung giải quyết các câu hỏi nghiên cứu sau:

- **RQ1:** Trong môi trường xử lý video 4K với khối lượng dữ liệu pixel lớn, phiên bản Wasm Scalar mang lại hệ số tăng tốc (**Speedup factor**) như thế nào so với mã nguồn JavaScript truyền thống?
- **RQ2:** Việc áp dụng tập lệnh SIMD giúp tối ưu hóa thời gian xử lý khung hình ở mức độ nào đối với các phép toán xử lý ảnh cơ bản?
- **RQ3:** Khi triển khai đa luồng trong Wasm, hiệu năng thực tế có đạt được sự tăng trưởng tuyến tính hay bị giới hạn bởi chi phí quản lý luồng và băng thông bộ nhớ của trình duyệt?
- **RQ4:** Kịch bản kết hợp SIMD và Multi-threaded có thực sự giúp kiểm soát độ trễ ổn định và ngăn chặn tình trạng mất đồng nhất (jitter) trong suốt quá trình xử lý video dài hay không?

1.4. Phạm vi và Phương pháp nghiên cứu

Để đảm bảo tính tập trung và độ tin cậy của các kết quả thực nghiệm, nghiên cứu này được thiết lập trong một khuôn khổ kỹ thuật và quy trình thực nghiệm chặt chẽ.

1.4.1. Phạm vi và Giới hạn của nghiên cứu

- **Phạm vi về kỹ thuật thực thi:** Nghiên cứu tập trung hoàn toàn vào khả năng tính toán tổng quát của CPU thông qua Wasm và JavaScript. Đề tài chủ động loại trừ các giải pháp tăng tốc phần cứng dựa trên GPU (như WebGL hay WebGPU) để cô lập và đánh giá chính xác hiệu quả tối ưu hóa SIMD và đa luồng của kiến trúc Wasm.
- **Phạm vi về dữ liệu thực nghiệm:** Để tạo ra áp lực đủ lớn nhằm bộc lộ các nút thắt cổ chai về hiệu năng, nghiên cứu sử dụng luồng dữ liệu video độ phân giải 4K UHD (3840×2160). Đây là ngưỡng dữ liệu mà các cơ chế thông dịch truyền thống của JavaScript thường gặp khó khăn trong việc duy trì tốc độ xử lý thời gian thực.
- **Phạm vi về thuật toán:** Đề tài triển khai một Pipeline Motion Detection tiêu chuẩn bao gồm ba giai đoạn xử lý pixel liên tục:
 1. *Chuyển đổi ảnh sang thang độ xám (Grayscale conversion):* Thao tác trên từng pixel đơn lẻ.
 2. *Làm mờ (Box Blur):* Thao tác tính toán tích chập trên vùng lân cận, đòi hỏi truy xuất bộ nhớ cao.
 3. *So sánh khung hình (Frame differencing):* So sánh sự khác biệt giữa các frame liên tiếp để xác định chuyển động.

1.4.2. Phương pháp nghiên cứu

Đề tài áp dụng **Phương pháp thực nghiệm hệ thống (Systematic Experimental Method)** để thu thập dữ liệu định lượng và phân tích hiệu năng dựa trên mô hình các biến số được kiểm soát chặt chẽ:

- **Thiết kế mô hình thực nghiệm:** Xây dựng một hệ thống Benchmark thống nhất trên trình duyệt (đại diện là Google Chrome) để thực thi cùng một thuật toán bằng cả JavaScript và Wasm dưới các kịch bản tối ưu hóa khác nhau.
- **Các biến số nghiên cứu:**
 - *Biến độc lập (Independent Variables):* Các kịch bản thực thi bao gồm JavaScript Baseline với:
 - (1) Wasm Scalar
 - (2) Wasm SIMD
 - (3) Wasm Multi-threaded
 - (4) Wasm SIMD + Multi-threaded.
 - *Biến phụ thuộc (Dependent Variables):* Các chỉ số hiệu năng định lượng bao gồm:
 - ◆ **Thời gian thực thi (Latency):** Tính bằng mili giây cho mỗi khung hình.

- ◆ **Tốc độ khung hình (FPS):** Khả năng duy trì sự mượt mà của pipeline.
- ◆ **Tính ổn định (P99 & Variance):** Sử dụng phân vị P99 và độ lệch chuẩn để đánh giá hiện tượng Jitter, yếu tố then chốt trong các hệ thống thời gian thực.
- ◆ **Hệ số tăng tốc (Speedup):** So sánh tương quan giữa các kịch bản tối ưu với Baseline.
- **Quy trình thu thập và xử lý dữ liệu:** Mỗi kịch bản được thực thi lặp lại trên một số lượng khung hình đủ lớn để loại bỏ các sai số ngẫu nhiên từ hệ điều hành. Dữ liệu thu thập được sẽ được phân tích thống kê để rút ra các kết luận khoa học về điểm bão hòa hiệu năng và các điểm đánh đổi kỹ thuật.

1.5. Đóng góp và Cấu trúc báo cáo

1.5.1. Đóng góp về mặt khoa học và thực tiễn

Kết quả của đề tài này mang lại những giá trị đóng góp cụ thể sau:

- **Về mặt khoa học:**
 - **Bộ dữ liệu thực nghiệm hệ thống:** Cung cấp các số liệu định lượng về hiệu năng của Wasm trong điều kiện tải cực lớn. Khác với các nghiên cứu trước đây tập trung vào ảnh tĩnh hoặc các thuật toán rời rạc, nghiên cứu này cung cấp cái nhìn toàn diện về sự thay đổi của các chỉ số Latency, FPS và đặc biệt là độ ổn định (P99, Jitter) trong một pipeline xử lý thực tế.
 - **Phân tích về điểm bão hòa hiệu năng:** Đây là đóng góp quan trọng nhất của đề tài. Nghiên cứu chỉ ra liệu việc kết hợp đồng thời SIMD và Multi-threaded có tạo ra hiệu ứng cộng hưởng tuyến tính hay không, hay sẽ đạt tới điểm tối hạn do các rào cản về băng thông bộ nhớ và chi phí quản lý luồng trên trình duyệt.
- **Về mặt thực tiễn:**
 - **Hệ thống Benchmark chuẩn hóa:** Xây dựng một framework đo lường mã nguồn mở, cho phép các nhà phát triển Web Media dễ dàng kiểm thử và so sánh các thuật toán xử lý video của họ trên các môi trường trình duyệt khác nhau.
 - **Khuyến nghị kỹ thuật:** Đưa ra các chỉ dẫn cho việc tối ưu hóa ứng dụng web hiệu năng cao. Kết quả nghiên cứu giúp lập trình viên xác định được kịch bản tối ưu nhất cho từng loại công việc xử lý pixel, tránh việc áp dụng đa luồng một cách lạm dụng khi không mang lại hiệu quả tương xứng với chi phí tài nguyên.

1.5.2. Cấu trúc báo cáo

Nội dung báo cáo được tổ chức thành 5 chương chính như sau:

- **Chương 1 – Giới thiệu:** Trình bày bối cảnh nghiên cứu, xác định rào cản của JavaScript và hứa hẹn của Wasm. Xác lập mục tiêu, phạm vi và câu hỏi nghiên cứu.
- **Chương 2 – Cơ sở lý thuyết:** Hệ thống hóa các kiến thức về kiến trúc Wasm, các đặc tả SIMD, Multi-threaded (SharedArrayBuffer/Web Workers). Giới thiệu về lớp bài toán Motion Detection và các phương pháp xử lý ảnh liên quan.
- **Chương 3 – Thiết kế và Cài đặt hệ thống:** Mô tả kiến trúc của hệ thống Benchmark, quy trình biên dịch từ C++ sang Wasm bằng Emscripten. Chi tiết hóa việc cài đặt pipeline xử lý video cho 4 kịch bản thực nghiệm: JavaScript vs Wasm Scalar, SIMD, Multi-threaded và SIMD + Multi-threaded.
- **Chương 4 – Kết quả thực nghiệm và Phân tích:** Trình bày các dữ liệu thu thập được qua biểu đồ và bảng biểu. Phân tích sâu về hệ số tăng tốc, hiện tượng Jitter và đánh giá mức độ ảnh hưởng của các kỹ thuật tối ưu hóa lên trải nghiệm video 4K.
- **Chương 5 – Kết luận và Hướng phát triển:** Tổng kết các kết quả đạt được đối chiếu với câu hỏi nghiên cứu ban đầu. Nêu rõ các hạn chế còn tồn tại và đề xuất các hướng nghiên cứu mở rộng trong tương lai (như WebGPU hoặc ứng dụng AI).

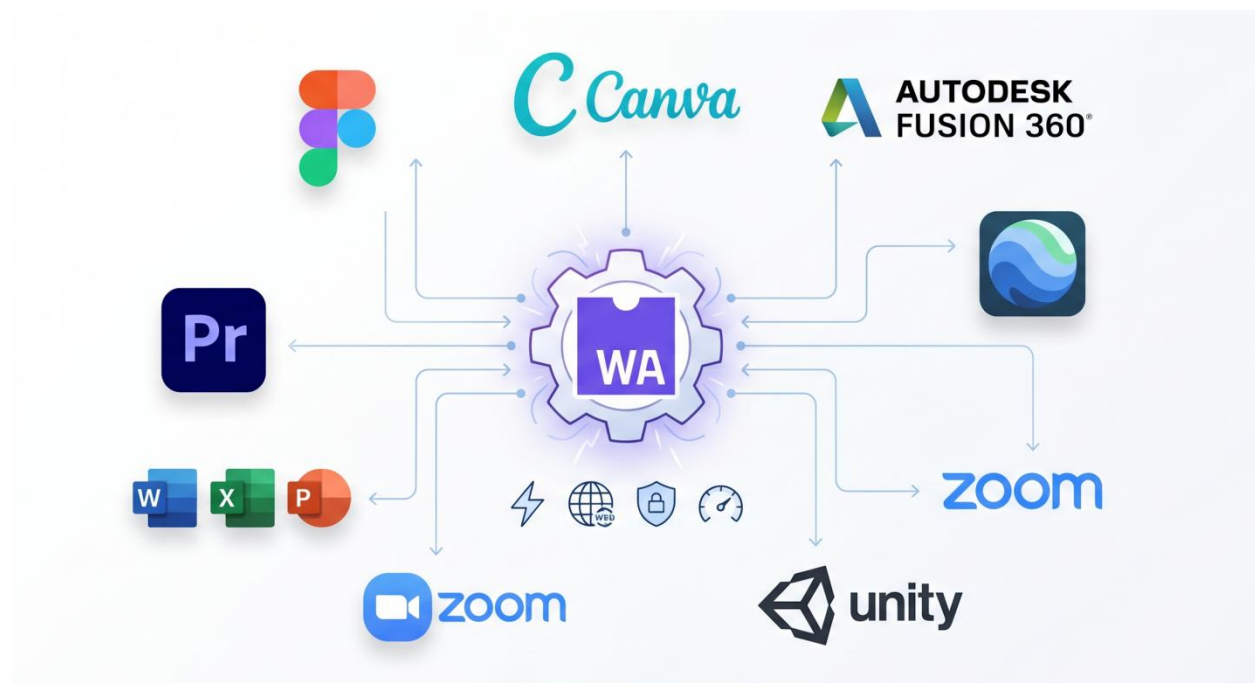
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về tính toán hiệu năng cao trên trình duyệt

2.1.1. Sự tiến hóa của Web Runtime

Trong kỷ nguyên sơ khai, trình duyệt web chủ yếu đóng vai trò hiển thị nội dung tĩnh. Tuy nhiên, sự bùng nổ của các ứng dụng trang đơn (SPA - Single Page Application) đã thúc đẩy việc chuyển dịch các tác vụ xử lý từ phía máy chủ về phía máy khách. Xu hướng này mang lại lợi ích về việc giảm tải cho máy chủ, tối ưu độ trễ mạng và tăng cường bảo mật dữ liệu người dùng.

Hiện nay, trình duyệt web hiện đại không chỉ dừng lại ở các tương tác UI đơn giản mà đã trở thành một nền tảng thực thi cho các tác vụ tính toán nặng như mô phỏng 3D, xử lý tín hiệu số, trí tuệ nhân tạo và đặc biệt là xử lý video độ phân giải cao trong thời gian thực. Điều này đặt ra yêu cầu cấp thiết về một môi trường thực thi có hiệu năng gần với ứng dụng gốc.



Hình 2.1 Hệ sinh thái WebAssembly

2.1.2. JavaScript và giới hạn hiệu năng cố hữu

JavaScript là ngôn ngữ chủ đạo trên trình duyệt, vận hành dựa trên cơ chế biên dịch vừa chạy vừa tối ưu. Mặc dù các công cụ như V8 (Chrome) đã được tối ưu hóa mạnh mẽ, JavaScript vẫn gặp phải ba rào cản hiệu năng chính khi đối mặt với dữ liệu lớn:

- **Hệ thống kiểu động (Dynamic Typing):** Trình duyệt phải thực hiện quy trình "vừa chạy vừa dò" để xác định kiểu dữ liệu của biến. Khi một giá trị về kiểu bị

sai (ví dụ: một hàm đang nhận số nguyên đột ngột nhận vào chuỗi), trình duyệt phải thực hiện quá trình Deoptimization, gây sụt giảm hiệu năng đột ngột và mất ổn định khung hình.

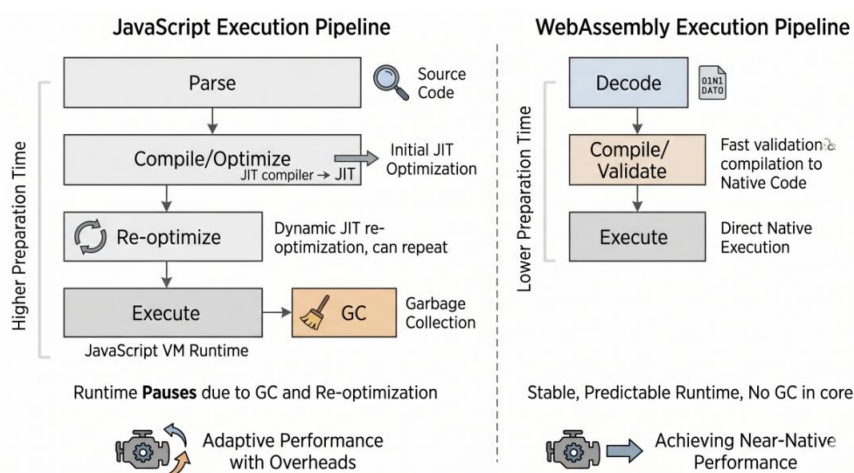
- **Quản lý bộ nhớ tự động:** Cơ chế quản lý bộ nhớ tự động của JavaScript giúp lập trình viên tránh rò rỉ bộ nhớ nhưng lại gây ra các khoảng dừng không thể dự đoán trước. Trong xử lý video 60 FPS, chỉ một khoảng dừng 16.7ms cũng đủ gây ra hiện tượng giật khung hình.
- **Hạn chế khai thác phần cứng:** JavaScript thiếu khả năng kiểm soát trực tiếp tập lệnh SIMD và mô hình đa luồng chia sẻ bộ nhớ thực thụ, dẫn đến chi phí sao chép dữ liệu cực lớn khi xử lý các mảng pixel khổng lồ.

2.1.3. WebAssembly – Giải pháp thực thi hiệu năng cao

WebAssembly ra đời như một chuẩn mở (W3C) nhằm hỗ trợ cho JavaScript trong các kịch bản đòi hỏi tốc độ xử lý tối đa.

Kiến trúc cốt lõi và Ưu thế:

- **Định dạng nhị phân và AOT:** Wasm là mã nhị phân đã được biên dịch sẵn (Ahead-of-Time - AOT) từ các ngôn ngữ kiểu tĩnh như C++, Rust. Trình duyệt có thể giải mã Wasm nhanh hơn nhiều lần so với việc phân tích mã nguồn JavaScript.
- **Máy ảo dựa trên ngăn xếp (Stack-based VM):** Wasm hoạt động với hệ thống kiểu tĩnh rõ rệt (i32, i64, f32, f64), cho phép ánh xạ trực tiếp sang mã máy mà không cần suy luận kiểu, giảm rủi ro Deoptimization do Wasm có kiểu tĩnh và pipeline biên dịch khác JavaScript JIT.
- **Bộ nhớ tuyến tính (Linear Memory):** Wasm sử dụng một mảng byte liên tục được quản lý thủ công (malloc/free). Điều này cho phép tái sử dụng vùng đệm qua các khung hình video, loại bỏ áp lực lên việc quản lý bộ nhớ và đảm bảo tính xác định về thời gian xử lý.



Hình 2.2 Pipeline thực thi của JavaScript JIT vs. WebAssembly

Bảng 2.1 Bảng so sánh quy trình thực thi giữa JavaScript và WebAssembly:

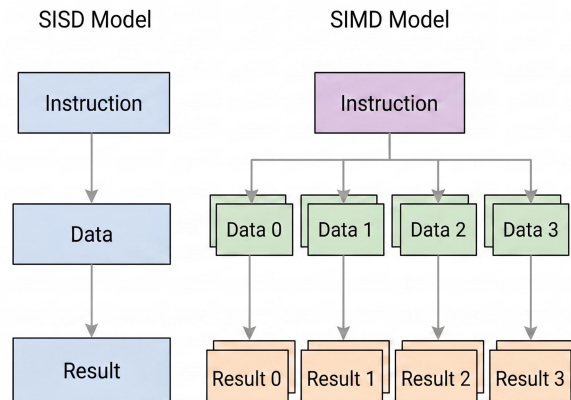
Giai đoạn	JavaScript (JIT)	WebAssembly (AOT)
Phân tích	Chuyển code thành AST (Chậm)	Đọc mã nhị phân (Nhanh)
Biên dịch	Vừa chạy vừa tối	Biên dịch trực tiếp sang mã máy
Thực thi	Có thể bị Deoptimization bất cứ lúc nào	Hiệu năng ổn định, tiệm cận ứng dụng gốc
Bộ nhớ	Quản lý tự động, có thể phát sinh GC pause	Linear Memory, cho phép quản lý chủ động hơn trong mã C/C++

2.2. Các kỹ thuật tối ưu hóa song song trên kiến trúc CPU hiện đại

2.2.1. Song song mức dữ liệu (SIMD)

Theo phân loại của Michael Flynn, SIMD (Single Instruction, Multiple Data) là mô hình kiến trúc máy tính cho phép một lệnh duy nhất thực hiện tính toán đồng thời trên nhiều phần tử dữ liệu. Trong xử lý video, mỗi pixel thường được xử lý độc lập, khiến bài toán này trở nên đặc biệt phù hợp với mô hình SIMD thay vì mô hình SISD (Single Instruction, Single Data) truyền thống.

- **WebAssembly SIMD 128-bit:** Được chuẩn hóa để cung cấp các thanh ghi vector 128-bit (kiểu dữ liệu v128). Đối với xử lý ảnh, thanh ghi này có thể chứa đồng thời 16 số nguyên 8-bit, cho phép tăng tốc độ xử lý pixel lên gấp nhiều lần trong một chu kỳ lệnh.
- **Lợi thế so với JavaScript:** Mặc dù các trình biên dịch JIT hiện đại (như V8) cố gắng thực hiện tự động vector hóa (auto-vectorization), nhưng cơ chế này thường không ổn định và khó dự đoán. Wasm cung cấp các hàm nội tại như `wasm_simd128.h`, cho phép lập trình viên kiểm soát trực tiếp phần cứng, đảm bảo hiệu năng tối ưu và nhất quán trên các trình duyệt. Các nghiên cứu cho thấy SIMD có thể cải thiện hiệu năng xử lý ảnh (như Box Blur) lên từ 3 đến 10 lần tùy thuộc vào kiến trúc phần cứng.

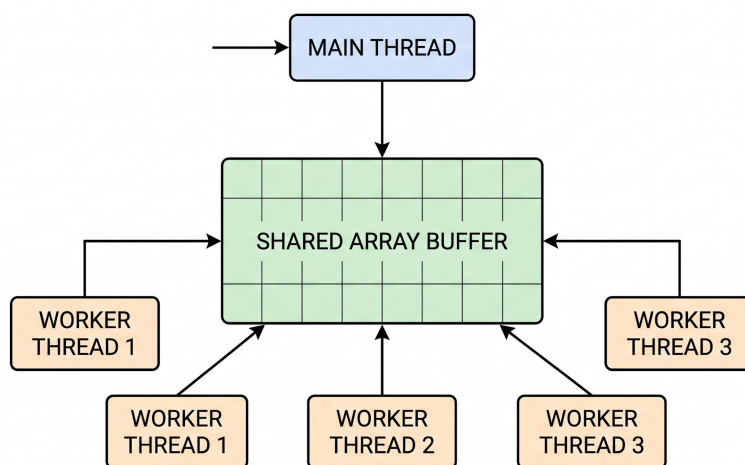


Hình 2.3 Mô hình SISD vs Mô hình SIMD

2.2.2. Song song mức luồng (Multi-threaded)

Nếu SIMD tối ưu hóa bên trong một lõi CPU, thì đa luồng (Multi-threaded) khai thác sức mạnh của nhiều lõi (multi-core).

- **Mô hình chia sẻ bộ nhớ:** Trên trình duyệt, tính năng này được hiện thực hóa thông qua SharedArrayBuffer (SAB). SAB cho phép nhiều Web Workers truy cập chung vào một vùng nhớ tuyến tính của Wasm mà không cần sao chép dữ liệu, giúp loại bỏ chi phí truyền dữ liệu cực lớn của JavaScript truyền thống.
- **Emscripten pthreads:** Wasm hỗ trợ thư viện **pthreads** chuẩn POSIX thông qua việc ánh xạ mỗi pthread thành một Web Worker. Cơ chế này cho phép các ứng dụng C++ đa luồng hiện có được biên dịch sang Web mà vẫn giữ nguyên logic song song đó, giúp khai thác tối đa tài nguyên CPU đa lõi hiện đại.



Hình 2.4 Kiến trúc Đa luồng với SharedArrayBuffer

2.2.3. Các rào cản vật lý và giới hạn hiệu năng thực tế

Việc áp dụng song song hóa không mang lại lợi ích tuyến tính vô hạn do các giới hạn vật lý của hệ thống:

- **Chi phí điều phối (Overhead):** Việc khởi tạo luồng, đồng bộ hóa và chia tách dữ liệu tiêu tốn một khoảng thời gian nhất định. Nếu khối lượng tính toán quá nhỏ, chi phí điều phối có thể lớn hơn lợi ích mang lại.
- **Giới hạn băng thông bộ nhớ (Memory-bound vs Compute-bound):** Xử lý video 4K đòi hỏi lưu lượng dữ liệu cực lớn (~33MB mỗi khung hình). Khi tốc độ tính toán của CPU vượt quá tốc độ cung cấp dữ liệu của RAM, bài toán sẽ chuyển từ giới hạn tính toán sang giới hạn băng thông bộ nhớ. Lúc này, việc thêm luồng hay lệnh SIMD sẽ không còn giúp tăng tốc đáng kể.
- **Rào cản an ninh:** Các lỗ hổng như Spectre đã khiến các trình duyệt áp dụng các biện pháp bảo mật nghiêm ngặt (như COOP/COEP), yêu cầu cấu hình máy chủ phức tạp để có thể kích hoạt SharedArrayBuffer và đa luồng.

2.3. Thuật toán xử lý video thời gian thực

2.3.1. Đặc điểm dữ liệu video 4K và áp lực tính toán

Trong nghiên cứu này, đối tượng thực nghiệm là video độ phân giải 4K UHD (3840 x 2160 pixels). Mỗi khung hình bao gồm khoảng 8,3 triệu điểm ảnh. Với định dạng màu RGBA (4 byte/pixel), dung lượng dữ liệu thô của một khung hình xấp xỉ 33,18MB.

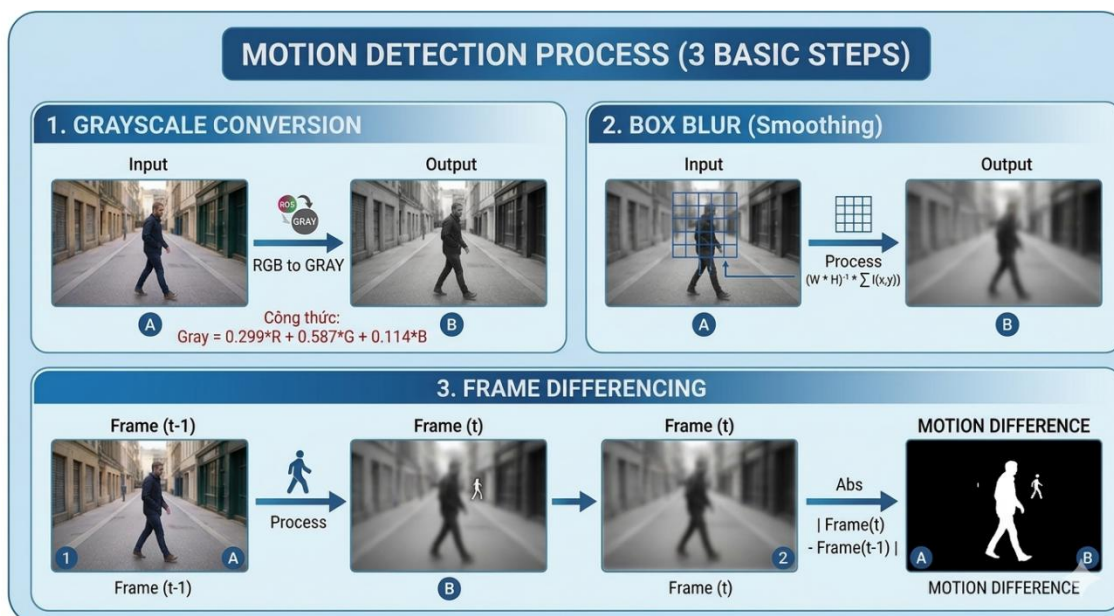
Để đảm bảo trải nghiệm mượt mà ở mức 60 FPS, hệ thống chỉ có tối đa 16,67 ms để hoàn thành toàn bộ chu trình xử lý, bao gồm cả việc giải mã, thực thi thuật toán và kết xuất dữ liệu. Đây là một áp lực cực lớn đối với JavaScript, đặc biệt là khi bài toán phát hiện chuyển động yêu cầu hàng trăm triệu phép tính trên mỗi khung hình.

2.3.2. Quy trình phát hiện chuyển động

Đề tài lựa chọn phương pháp **so sánh khung hình (Frame Differencing)** kết hợp với **bộ lọc mờ (Box Blur)** làm đối tượng nghiên cứu. Thuật toán này có tính chất song song cực cao, cho phép mỗi điểm ảnh hoặc khối điểm ảnh có thể được xử lý độc lập, lý tưởng để đánh giá khả năng khai thác SIMD và đa luồng.

Quy trình xử lý bao gồm ba giai đoạn chính:

1. **Chuyển đổi ảnh sang thang độ xám (Grayscale Conversion):** Giảm chiều dữ liệu.
2. **Làm mờ (Box Blur):** Loại bỏ nhiễu kỹ thuật số để tránh phát hiện sai.
3. **So sánh khung hình (Frame Differencing):** Xác định các pixel có sự biến đổi cường độ sáng vượt ngưỡng.



Hình 2.5 Pipeline Thuật toán Phát hiện Chuyển động

2.3.3. Các bước xử lý chi tiết và kỹ thuật tối ưu hóa

a. Chuyển đổi ảnh xám

Mục đích là chuyển dữ liệu từ 4 kênh màu (RGBA) về 1 kênh cường độ sáng duy nhất nhằm giảm 75% khối lượng dữ liệu cho các bước sau. Báo cáo sử dụng tiêu chuẩn ITU-R BT.601 với công thức:

$$Y = 0.299R + 0.587G + 0.114B$$

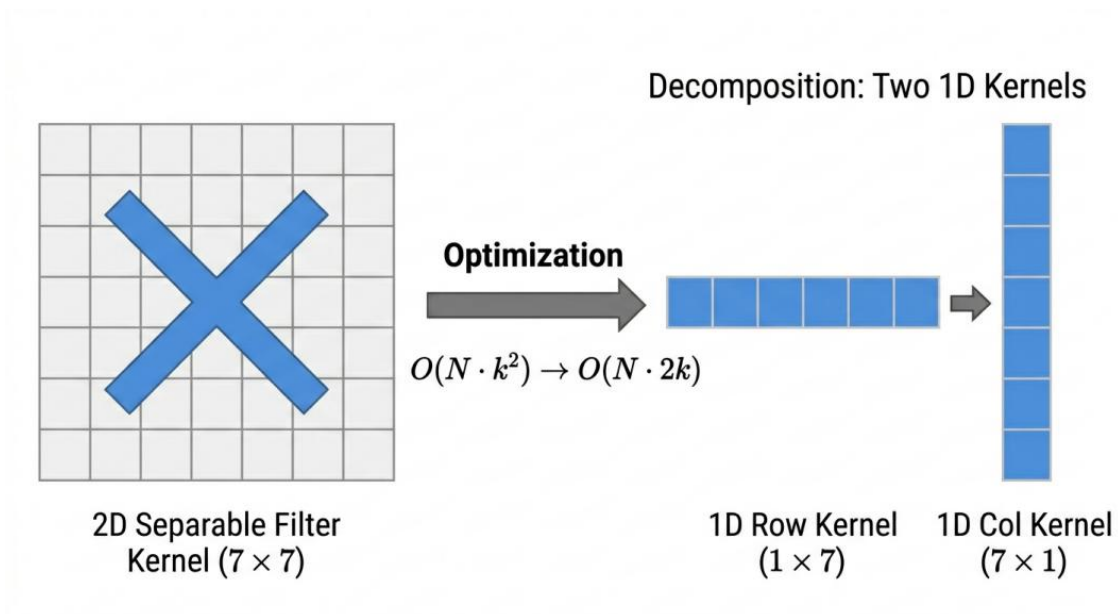
Để tối ưu hóa cho CPU và tận dụng đơn vị xử lý số nguyên (ALU), công thức trên được chuyển đổi sang dạng phép nhân số nguyên và trượt bit:

$$Y = (77R + 150G + 29B) \gg 8$$

b. Lọc nhiễu Box Blur - Nút cổ chai hệ thống

Đây là giai đoạn tốn tài nguyên nhất, chiếm khoảng 90% khối lượng tính toán của toàn bộ pipeline.

- **Cài đặt thông thường:** Với mỗi pixel, hệ thống tính trung bình cộng của các pixel trong vùng lân cận $k \times k$ ($k=7$). Độ phức tạp là $O(Nk^2)$, tương đương hơn 400 triệu phép tính cho mỗi khung hình 4K.
- **Cài đặt tối ưu (Separable Filter & Sliding Window):**
 - Tận dụng tính chất có thể tách rời của bộ lọc Box Blur để chia một phép tích chập 2D thành hai lượt xử lý 1D (ngang và dọc).
 - Kết hợp với kỹ thuật cửa sổ trượt (Sliding Window), độ phức tạp được giảm xuống mức tuyến tính $O(N)$, giúp tiết kiệm khoảng 24,5 lần số phép tính so với cách làm thông thường.



Hình 2.6 Tối ưu hóa Box blur bằng Separable Filter

c. So sánh khung hình - Frame Differencing

Pixel tại vị trí (x, y) được xác định là có chuyển động nếu chênh lệch tuyệt đối về cường độ sáng giữa khung hình hiện tại (B_t) và khung hình trước đó (B_{t-1}) vượt qua một ngưỡng T xác định (trong thực nghiệm $T=30$):

$$M(x, y) = |B_t(x, y) - B_{t-1}(x, y)| > T$$

Kết quả cuối cùng được ghi trực tiếp vào vùng nhớ đệm RGBA để hiển thị lên trình duyệt qua Canvas API.

2.4. Khung đánh giá hiệu năng

Để đánh giá sự khác biệt giữa các phương pháp triển khai một cách khách quan và khoa học, báo cáo sử dụng một tập hợp các chỉ số định lượng tập trung vào hai khía cạnh: **tốc độ xử lý và tính ổn định của hệ thống thời gian thực.**

2.4.1. Các độ đo định lượng cốt lõi

- **Thời gian xử lý (Latency):** Đây là khoảng thời gian từ khi bắt đầu đến khi hoàn thành pipeline xử lý của duy nhất một khung hình, đơn vị tính bằng mili giây. Trong môi trường trình duyệt, độ trễ này được đo bằng API `performance.now()` với độ chính xác mức micro-giây. Ngưỡng mục tiêu cho video 60 FPS là Latency < 16,67ms.
- **Thông lượng khung hình (FPS - Frames Per Second):** Số lượng khung hình được hệ thống xử lý hoàn chỉnh và hiển thị trong một giây. Chỉ số này phản ánh hiệu năng tổng thể của ứng dụng, bao gồm cả chi phí giải mã video và vẽ lên màn hình (Canvas).
- **Độ lệch chuẩn (Standard Deviation - σ):** Đo lường mức độ biến động của thời gian xử lý (Jitter). Một giá trị σ thấp cho thấy hệ thống vận hành ổn định, trong khi σ cao thường là dấu hiệu của hiện tượng "đóng băng" do bộ quản lý bộ nhớ tự động hoặc cơ chế điều phối luồng không tối ưu.

2.4.2. Các thông số thống kê chuyên sâu

Do môi trường trình duyệt chịu ảnh hưởng bởi nhiều yếu tố ngẫu nhiên (tác vụ nền của hệ điều hành, cơ chế tiết kiệm năng lượng), việc chỉ sử dụng **giá trị trung bình (Mean)** có thể dẫn đến kết quả sai lệch. Báo cáo bổ sung các chỉ số:

- **Trung vị (Median):** Phản ánh hiệu năng trong điều kiện "điển hình", giúp loại bỏ ảnh hưởng của các giá trị bất thường.
- **Phân vị (Percentile) P99:** Cho biết ngưỡng thời gian mà 99% số khung hình được xử lý nhanh hơn giá trị đó. Trong xử lý thời gian thực, P99 là chỉ số then chốt; nếu P99 vượt quá 16,67 ms, người dùng sẽ cảm nhận được hiện tượng giật hình ngay cả khi Latency trung bình thấp.

2.4.3. Các mô hình lý thuyết so sánh

Để đánh giá mức độ khai thác tài nguyên phần cứng của các kịch bản tối ưu, báo cáo áp dụng hai mô hình toán học:

- **Tỉ lệ tăng tốc (Speedup - S):** Đo lường mức độ cải thiện hiệu năng của phương pháp tối ưu ($T_{\text{optimized}}$) so với phương pháp cơ sở (T_{baseline}):

$$S = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}$$

- **Định luật Amdahl:** Dùng để dự đoán giới hạn tăng tốc lý thuyết khi song song hóa một chương trình có tỷ lệ phần code có thể song song hóa p trên n luồng:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

Mô hình này giúp xác định xem việc thêm luồng (Multi-threaded) có thực sự mang lại lợi ích tương xứng hay hệ thống đã chạm ngưỡng giới hạn của phần mã xử lý tuần tự.

2.5. Tổng quan các công trình nghiên cứu liên quan

Việc nghiên cứu hiệu năng của Wasm so với JavaScript đã thu hút được sự quan tâm đáng kể từ cộng đồng nghiên cứu khoa học máy tính từ khi Wasm được chuẩn hóa vào năm 2017.

2.5.1. Các nghiên cứu so sánh hiệu năng thực thi cơ bản

Nhiều nghiên cứu tiền đề đã xác nhận lợi thế về tốc độ của Wasm nhờ vào định dạng nhị phân và cơ chế biên dịch AOT. Nghiên cứu của Haas và các cộng sự (2017) đã đặt nền móng khi chứng minh Wasm có thể đạt tốc độ thực thi tiệm cận ứng dụng gốc, thường chỉ chậm hơn từ 1,1 đến 2 lần so với mã C tương ứng.

Tuy nhiên, nghiên cứu của Jangda và các cộng sự (2019) trong bài báo "*Not So Fast: Analyzing the Performance of WebAssembly vs Native Code*" đã đưa ra một cái nhìn khắt khe hơn khi chỉ ra rằng trong thực tế, Wasm vẫn chậm hơn mã gốc trung bình khoảng 45% đến 55% lần do các rào cản về an ninh và quản lý bộ nhớ tuyến tính. Các nghiên cứu này cũng đồng thuận rằng JavaScript gặp khó khăn trong các tác vụ nặng do hiện tượng sụt giảm hiệu năng khi dự đoán sai kiểu dữ liệu (deoptimization) và ảnh hưởng của bộ quản lý bộ nhớ tự động.

2.5.2. WebAssembly trong lĩnh vực thị giác máy tính và xử lý ảnh

Việc đưa các thư viện nặng như OpenCV lên web thông qua dự án OpenCV.js đã chứng minh tính khả thi của việc xử lý thị giác máy tính trực tiếp trên trình duyệt. Nghiên cứu về hiệu suất của các thuật toán tích chập – tương tự như bộ lọc Box Blur trong báo

cáo này – cho thấy Wasm mang lại sự ổn định vượt trội so với JavaScript nhờ quản lý bộ nhớ thủ công.

Đặc biệt, bài báo "Performance Evaluation of Image Convolution with WebAssembly" đã chỉ ra rằng khi xử lý các ma trận điểm ảnh lớn, Wasm giúp giảm đáng kể thời gian trễ so với các phương pháp dựa trên mảng định kiểu (TypedArray) của JavaScript.

2.5.3. Đánh giá các kỹ thuật tối ưu hóa phần cứng (SIMD và Multi-threaded)

Sự ra đời của các đặc tả Wasm hiện đại đã cho phép khai thác trực tiếp tập lệnh SIMD 128-bit và đa luồng qua pthreads. Các nghiên cứu hiện tại thường tập trung vào các bài kiểm tra vi mô (micro-benchmarks) để đo lường mức tăng tốc (speedup) lý thuyết của từng kỹ thuật riêng lẻ.

Nghiên cứu về SIMD trong Wasm chỉ ra rằng việc vector hóa có thể mang lại hiệu năng cao gấp 3 đến 8 lần cho các tác vụ xử lý pixel song song. Đồng thời, các nghiên cứu về đa luồng dựa trên SharedArrayBuffer nhấn mạnh khả năng loại bỏ chi phí sao chép dữ liệu giữa các Web Worker, giúp tối ưu hóa luồng dữ liệu cho các ứng dụng thời gian thực.

2.5.4. Khoảng trống nghiên cứu và đóng góp của báo cáo

Mặc dù có nhiều nghiên cứu về từng kỹ thuật tối ưu riêng biệt, nhưng vẫn tồn tại một **khoảng trống nghiên cứu** đáng kể:

- **Thiếu các đánh giá tích lũy:** Các nghiên cứu hiện tại chủ yếu tập trung vào SIMD hoặc Multi-threaded đơn lẻ mà chưa phân tích sâu hiệu quả khi kết hợp đồng thời cả hai kỹ thuật này (kịch bản tối ưu tích lũy).
- **Thiếu thực nghiệm trên dữ liệu 4K thực tế:** Phần lớn các benchmark sử dụng các tập dữ liệu nhỏ hoặc video độ phân giải thấp, vốn chưa tạo đủ áp lực để bộc lộ rõ rệt hiện tượng giới hạn băng thông bộ nhớ (memory-bound) trên trình duyệt.

Báo cáo này tập trung giải quyết các khoảng trống trên bằng cách xây dựng hệ thống benchmark có kiểm soát, đánh giá trực tiếp trên video 4K thông qua 4 kịch bản tối ưu hóa lũy tiến. Kết quả nghiên cứu sẽ cung cấp dữ liệu định lượng về điểm tới hạn giữa tốc độ tính toán và băng thông bộ nhớ trong môi trường Web hiện đại.

CHƯƠNG 3: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

3.1 Mục tiêu và Nguyên tắc thiết kế

3.1.1 Mục tiêu thiết kế hệ thống

Mục tiêu cốt lõi của việc thiết kế hệ thống là xây dựng một môi trường thực nghiệm trung lập và chuẩn hóa nhằm đánh giá định lượng sự khác biệt về hiệu năng giữa JavaScript và Wasm trong bài toán xử lý video thời gian thực. Hệ thống không tập trung vào việc xây dựng một ứng dụng hoàn thiện cho người dùng cuối mà ưu tiên tính chính xác trong đo lường và khả năng tái lập kết quả.

Hệ thống được thiết kế theo mô hình **ứng dụng đơn trang** (Single Page Application), thực thi hoàn toàn tại phía máy khách. Việc lựa chọn kiến trúc này giúp đảm bảo rằng kết quả đo lường phản ánh trực tiếp năng lực tính toán của trình duyệt và tài nguyên phần cứng, loại bỏ hoàn toàn các yếu tố gây nhiễu từ độ trễ mạng hay cấu hình máy chủ.

3.1.2 Nguyên tắc đảm bảo tính khoa học

Để đảm bảo kết quả so sánh giữa các kịch bản tối ưu là khách quan và có giá trị học thuật, hệ thống tuân thủ nghiêm ngặt các nguyên tắc thiết kế sau:

- **Cô lập môi trường:** Mỗi kịch bản tối ưu (từ Scalar đến SIMD và Multi-threaded) được thực thi trên các cổng HTTP riêng biệt (8080–8083). Điều này giúp ngăn chặn tình trạng chồng chéo trạng thái bộ nhớ đệm hoặc các tối ưu hóa tiềm ẩn của trình duyệt từ kịch bản trước làm ảnh hưởng đến kết quả của kịch bản sau.
- **Kiểm soát biến số:** Dữ liệu đầu vào, giao diện đo lường, thời lượng benchmark và quy trình thu thập số liệu được giữ thống nhất giữa các kịch bản. Pipeline xử lý ở mức chức năng đều gồm ba bước chính: chuyển ảnh xám, làm mờ và so sánh khung hình. Tuy nhiên, ở các kịch bản tối ưu cao hơn, một số bước xử lý được thay đổi về cách cài đặt nhằm đánh giá tác động của tối ưu hóa thuật toán, SIMD và đa luồng. Do đó, kết quả thực nghiệm phản ánh hiệu quả tổng hợp của từng chiến lược tối ưu hóa, không chỉ riêng sự khác biệt giữa JavaScript và WebAssembly.
- **Đo lường thời gian xử lý khung hình:** Hệ thống ghi nhận thời gian tại các điểm ngay trước và sau khi gọi hàm xử lý pixel. Cách đo này loại bỏ các chi phí nằm ngoài vòng xử lý như giải mã video và thao tác vẽ kết quả lên Canvas. Đối với JavaScript, chỉ số này phản ánh thời gian chạy hàm xử lý ảnh bằng JS. Đối với WebAssembly, chỉ số này bao gồm cả chi phí bridge như cấp phát bộ nhớ đầu vào, copy-in, thực thi C++, copy-out và giải phóng bộ nhớ.
- **Hạn chế phụ thuộc:** Hệ thống không sử dụng các thư viện bên thứ ba (như OpenCV.js) hoặc các framework hiện đại nhằm loại bỏ chi phí từ các lớp trừu tượng bổ sung, đảm bảo đo lường hiệu năng của mã nguồn thuần túy.

- **Tự động hóa và lặp lại:** Quy trình biên dịch và khởi chạy được tự động hóa qua các kịch bản lệnh nhằm giảm thiểu sai sót do thao tác thủ công, cho phép thực hiện đo lường liên tục qua hàng nghìn khung hình để lấy mẫu thống kê.

3.2 Kiến trúc hệ thống tổng thể

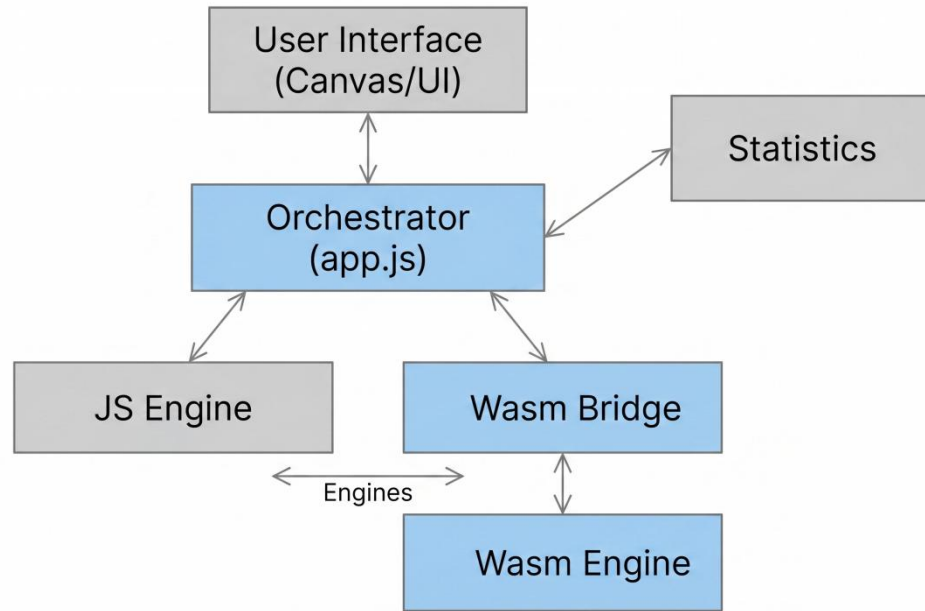
3.2.1 Mô hình Single Page Application (SPA)

Hệ thống được xây dựng theo mô hình Ứng dụng đơn trang (SPA), cho phép toàn bộ quá trình xử lý và tính toán diễn ra trực tiếp tại trình duyệt của người dùng. Trong kiến trúc này, máy chủ HTTP chỉ đóng vai trò phục vụ các tài nguyên tĩnh như mã nguồn HTML, JavaScript và các mô-đun nhị phân Wasm. Việc thực thi logic hoàn toàn tại phía máy khách giúp loại bỏ các biến số gây nhiễu từ độ trễ mạng hay cấu hình máy chủ, đảm bảo rằng các kết quả thực nghiệm phản ánh chính xác hiệu năng của các engine thực thi (JavaScript và Wasm) trên cùng một cấu hình phần cứng.

3.2.2 Sơ đồ khối các thành phần chính

Để đảm bảo tính linh hoạt cho việc triển khai 04 kịch bản tối ưu hóa khác nhau, hệ thống được cấu trúc thành các mô-đun chức năng tách biệt:

- **Giao diện người dùng:** Thành phần này cung cấp bộ công cụ tương tác, cho phép người dùng tải lên video 4K mẫu, điều khiển các phiên benchmark và theo dõi kết quả xử lý trực quan thông qua các thành phần Canvas.
- **Bộ điều phối:** Đóng vai trò là trung tâm điều khiển của hệ thống. Thành phần này quản lý vòng lặp xử lý chính (main loop), điều phối luồng dữ liệu pixel giữa các engine và kích hoạt quy trình thu thập Metrics tại các điểm chốt.
- **Engine xử lý (JavaScript & Wasm):** Chứa các cài đặt lõi của thuật toán phát hiện chuyển động. Engine JavaScript đóng vai trò là đường tham chiếu (baseline), trong khi Engine Wasm được biên dịch từ mã nguồn C++ để thực thi các kịch bản tối ưu phần cứng.
- **Cầu nối WebAssembly** Đây là lớp trung gian chuyên biệt chịu trách nhiệm quản lý bộ nhớ tuyến tính (Linear Memory). Nó thực hiện việc cấp phát vùng đệm và sao chép dữ liệu pixel giữa môi trường JavaScript (Browser Heap) và Wasm (Isolated Memory).
- **Mô-đun thống kê:** Chịu trách nhiệm lưu trữ các dấu thời gian thô và thực hiện phân tích thống kê sau khi kết thúc quá trình lấy mẫu để đưa ra các chỉ số như Mean, Median, P99 và Độ lệch chuẩn.



Hình 3.1 Sơ đồ khối kiến trúc hệ thống tổng thể

3.2.3 Luồng xử lý dữ liệu

Luồng dữ liệu trong hệ thống được thiết kế theo dạng đường ống (pipeline) để tối ưu hóa thông lượng xử lý cho video độ phân giải cao. Quy trình xử lý một khung hình bao gồm các giai đoạn sau:

1. **Trích xuất:** Bộ điều phối sử dụng *requestAnimationFrame* để đồng bộ với tần số quét của màn hình, trích xuất dữ liệu pixel thô từ khung hình video hiện tại thông qua Canvas API.
2. **Chuẩn bị:** Dữ liệu được chuyển đổi sang định dạng *Uint8ClampedArray*. Đối với Wasm, dữ liệu này sẽ được sao chép vào vùng nhớ tuyến tính thông qua Wasm Bridge.
3. **Thực thi:** Engine thực hiện chuỗi thuật toán tích lũy: Chuyển đổi ảnh xám -> Lọc nhiễu Box Blur -> So sánh chênh lệch khung hình.
4. **Kết xuất và Thu thập:** Kết quả xử lý được đẩy ngược lại Canvas để hiển thị, đồng thời thời gian thực thi của thuật toán được ghi nhận lại để phục vụ phân tích thống kê.



Hình 3.2 Luồng xử lý dữ liệu

3.3 Thiết kế tầng lưu trữ và Giao tiếp

Sự khác biệt về mô hình bộ nhớ giữa JavaScript (với cơ chế quản lý bộ nhớ tự động) và Wasm (với Linear Memory) đặt ra thách thức lớn về mặt hiệu năng khi xử lý dữ liệu video 4K. Tầng giao tiếp (The Bridge) được thiết kế để tối ưu hóa quá trình luân chuyển dữ liệu và giảm thiểu độ trễ nảy sinh từ việc sao chép vùng nhớ.

3.3.1 Quản lý bộ nhớ tuyến tính trong Wasm

Wasm tương tác với dữ liệu thông qua một vùng nhớ duy nhất được gọi là **Bộ nhớ tuyến tính (Linear Memory)**. Đây thực chất là một mảng byte liên tục mà cả JavaScript và Wasm đều có thể truy cập.

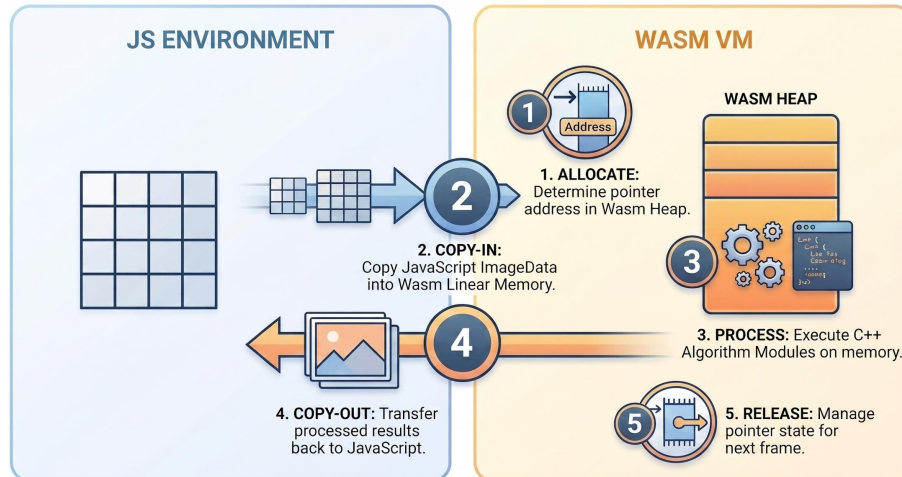
- **Cấu trúc lưu trữ:** Đối với video độ phân giải 4K, hệ thống cấp phát các vùng đệm cố định trong bộ nhớ tuyến tính để chứa: khung hình hiện tại, khung hình trước đó và khung hình kết quả.
- **Tối ưu hóa thủ công:** Thay vì dựa vào Garbage Collector của JavaScript — vốn gây ra các khoảng dừng không xác định — hệ thống thực hiện quản lý bộ nhớ thủ công qua malloc và free. Trong lỗi xử lý C++, các vùng đệm trung gian như ảnh xám, ảnh đã làm mờ và ảnh đầu ra được lưu trong các vector tái sử dụng giữa các khung hình. Cách này giúp giảm số lần cấp phát bộ nhớ bên trong module Wasm so với việc tạo mảng mới liên tục trong JavaScript. Tuy nhiên, ở tầng giao tiếp giữa JavaScript và Wasm, dữ liệu đầu vào vẫn cần được cấp phát, sao chép vào Linear Memory và kết quả vẫn cần được sao chép ngược ra JavaScript để hiển thị trên Canvas. Vì vậy, chi phí copy-in/copy-out vẫn là một thành phần quan trọng trong latency tổng thể của Wasm.

3.3.2 Cơ chế điều phối dữ liệu

Do tính chất cô lập của máy ảo Wasm, dữ liệu pixel từ đối tượng ImageData của trình duyệt không thể được xử lý trực tiếp bởi mã C++. Hệ thống triển khai chu trình điều phối 5 bước:

1. **Allocate:** Xác định địa chỉ con trỏ trong vùng nhớ Wasm Heap.
2. **Copy-in:** Sao chép dữ liệu của JavaScript vào bộ nhớ tuyến tính của Wasm.
3. **Process:** Thực thi các module thuật toán C++ trên vùng nhớ này.
4. **Copy-out:** Chuyển kết quả từ bộ nhớ tuyến tính ngược lại môi trường JavaScript để chuẩn bị hiển thị.
5. **Release:** Quản lý trạng thái con trỏ để chuẩn bị cho khung hình kế tiếp.

Lưu ý về hiệu năng: Với độ phân giải 4K, mỗi khung hình chiếm khoảng 33MB dữ liệu thô, chi phí sao chép này được tính gộp vào tổng độ trễ (latency) của Wasm để đảm bảo tính khách quan khi so sánh với JavaScript.



Hình 3.3 Cơ chế sao chép dữ liệu qua Cầu nối

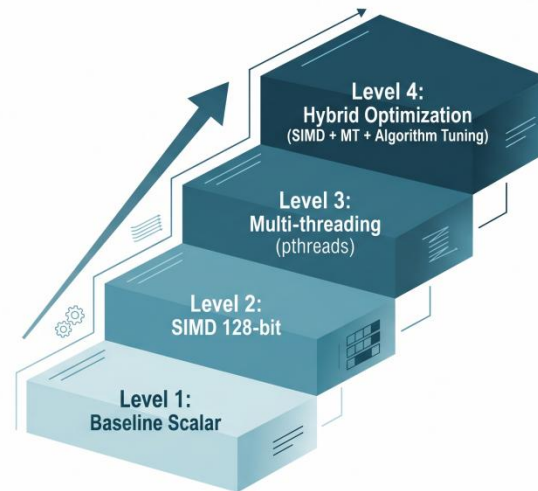
3.3.3 Đồng bộ hóa bộ nhớ trong đa luồng

Trong các kịch bản tối ưu hóa đa luồng, cơ chế giao tiếp được nâng cấp thông qua **SharedArrayBuffer**.

- **Chia sẻ bộ nhớ thực thụ:** Khác với cơ chế truyền thông điệp thông thường của Web Workers yêu cầu sao chép dữ liệu, SAB cho phép tất cả các luồng Worker truy cập đồng thời vào cùng một vùng nhớ tuyến tính của Wasm.
- **Đồng bộ hóa luồng:** Trong các kịch bản đa luồng, hệ thống sử dụng pthreads của Emscripten, ánh xạ các `std::thread` trong C++ sang Web Workers trên trình duyệt. Dữ liệu được chia thành các vùng xử lý độc lập để mỗi luồng ghi vào một phần riêng của bộ đệm, qua đó hạn chế tranh chấp dữ liệu. Sau mỗi giai đoạn xử lý, luồng chính thực hiện `join` để chờ các worker hoàn tất trước khi chuyển sang bước tiếp theo. Cách tiếp cận này dựa trên mô hình fork-join thay vì tự triển khai trực tiếp các lệnh `Atomics` ở tầng mã nguồn ứng dụng.
- **Yêu cầu bảo mật:** Để kích hoạt tầng giao tiếp này, máy chủ bắt buộc phải cấu hình các tiêu đề (header) bảo mật để trình duyệt cho phép sử dụng SAB:
 - **COOP (Cross-Origin-Opener-Policy):** Cô lập tab hiện tại với các tab hoặc cửa sổ khác. Đảm bảo trang web nằm trong một tiến trình riêng biệt, không để lộ thông tin cho các web khác.
 - **COEP (Cross-Origin-Embedder-Policy):** Kiểm soát các tài nguyên nhúng vào trang (như ảnh, video, script). Ngăn chặn việc vô tình nạp các dữ liệu nhạy cảm từ nguồn khác vào cùng một tiến trình đang dùng chung bộ nhớ.

3.4 Cài đặt các kịch bản thực nghiệm

Phần này mô tả chi tiết quá trình triển khai thuật toán phát hiện chuyển động qua bốn cấp độ tối ưu hóa khác nhau. Mỗi kịch bản được thiết kế để cô lập và đánh giá hiệu quả của một tầng công nghệ cụ thể.



Hình 3.4 Chiến lược tối ưu hóa tích lũy

3.4.1 Kịch bản 1: Triển khai Scalar - Baseline

Kịch bản này được thiết lập làm điểm tham chiếu cơ sở để đo lường lợi ích hiệu năng thuần túy của môi trường thực thi Wasm so với JavaScript.

- **Phía JavaScript:** Sử dụng kiểu dữ liệu số thực (float64) mặc định và thuật toán Box Blur nguyên bản với độ phức tạp $O(Nk^2)$. Điểm yếu cốt lõi ở đây là áp lực lên bộ quản lý bộ nhớ tự động do việc khởi tạo mảng kiểu *Uint8ClampedArray* lưu dữ liệu ảnh mới cho mỗi khung hình.
- **Phía WebAssembly:** Triển khai mã nguồn C++ tương đương nhưng tận dụng khả năng quản lý bộ nhớ thủ công. Hệ thống tái sử dụng vùng đệm, giúp loại bỏ hoàn toàn các khoảng dừng do cơ chế quản lý bộ nhớ tự động, đảm bảo tính xác định trong thời gian xử lý.

3.4.2 Kịch bản 2: Tối ưu hóa mức dữ liệu - SIMD 128-bit

Kịch bản này tập trung vào việc khai thác xử lý song song mức dữ liệu bằng SIMD 128-bit, đồng thời kết hợp một số tối ưu hóa thuật toán phù hợp với xử lý ảnh.

- **Chuyển đổi sang số nguyên:** Công thức ảnh xám được chuyển từ dạng số thực $Y = 0.299R + 0.587G + 0.114B$ sang dạng xấp xỉ số nguyên $Y = (77R + 150G + 29B) \gg 8$. Cách cài đặt này giảm chi phí tính toán dấu phẩy động, dù trong mã hiện tại bước chuyển ảnh xám vẫn được thực hiện bằng vòng lặp tuần tự.

- **Tối ưu hóa Box Blur:** Thay vì sử dụng tích chập 2D trực tiếp với kernel 7×7 , hệ thống áp dụng separable blur kết hợp sliding window. Nhờ đó, số phép cộng trên mỗi pixel giảm đáng kể so với cài đặt naive ở kịch bản Scalar.
- **Vector hóa bước so sánh khung hình:** SIMD 128-bit được sử dụng ở giai đoạn frame differencing và sinh dữ liệu RGBA đầu ra. Mỗi thanh ghi vector xử lý đồng thời 16 giá trị grayscale 8-bit. Mã sử dụng các phép toán max/min để tính trị tuyệt đối của chênh lệch, sau đó so sánh với ngưỡng và dùng bitmask để đếm số pixel thay đổi.

Vì vậy, mức tăng tốc của kịch bản SIMD đến từ sự kết hợp giữa tối ưu hóa thuật toán blur, chuyển đổi số nguyên và SIMD ở bước diff/output, không chỉ từ riêng tập lệnh SIMD.

3.4.3 Kịch bản 3: Tối ưu hóa mức luồng - Multi-threaded

Kịch bản này chuyển dịch trọng tâm từ việc giảm số lượng lệnh sang việc phân phối khối lượng tính toán trên các lõi CPU khác nhau thông qua mô hình Fork-Join.

- **Chiến lược phân mảnh:** Dữ liệu ảnh được chia thành các dải hàng ngang độc lập. Mỗi luồng Worker xử lý một dải dữ liệu riêng biệt để tránh tranh chấp bộ nhớ.
- **Đồng bộ hóa luồng:** Sử dụng thư viện pthreads của C++ được ánh xạ sang Web Workers thông qua cờ hiệu -pthread. Luồng chính thực hiện ba chu kỳ Fork-Join tương ứng với ba bước của thuật toán để đảm bảo tính toàn vẹn của dữ liệu giữa các giai đoạn.

3.4.4 Kịch bản 4: Tối ưu hóa tích lũy - SIMD + Multi-threaded

Đây là kịch bản tối ưu hóa cao cấp nhất, kết hợp đồng thời cả ba tầng: Thuật toán, Dữ liệu và Luồng.

- **Nâng cấp thuật toán:** Thay thế Box Blur nguyên bản bằng Separable Blur, chia phép tích chập 2D thành hai giai đoạn 1D (ngang và dọc). Việc này kết hợp với kỹ thuật cửa sổ trượt giúp giảm độ phức tạp tính toán xuống mức tối thiểu.
- **Hiệu ứng nhân:** Hệ thống thực thi tập lệnh SIMD bên trong từng luồng Worker. Pipeline xử lý được chia thành bốn giai đoạn Fork-Join, tận dụng tối đa băng thông bộ nhớ và tài nguyên đa lõi của CPU.
- **Căn chỉnh dữ liệu:** Để tối ưu hóa SIMD trong đa luồng, các khối dữ liệu được phân chia theo bội số của 16-byte, giúp loại bỏ phần dư xử lý tuần tự và đạt thông lượng tối đa.

3.5 Môi trường thực nghiệm

Để đảm bảo tính nhất quán và độ tin cậy của các dữ liệu thu thập được, toàn bộ các thí nghiệm trong báo cáo này được thực hiện trong một môi trường kiểm soát chặt chẽ với các thông số kỹ thuật cụ thể về phần cứng và phần mềm.

3.5.1 Cấu hình phần cứng

Thực nghiệm được tiến hành trên hệ thống máy tính cá nhân với cấu hình chi tiết như sau:

- **Bộ vi xử lý (CPU):** Apple M4. Cấu hình này hỗ trợ tập lệnh SIMD và có 10 nhân xử lý vật lý (bao gồm 4 nhân hiệu năng cao và 6 nhân tiết kiệm điện). Hệ thống cho phép đánh giá hiệu quả của đa luồng tối đa 4 luồng theo thiết kế, tương ứng với số lượng nhân hiệu năng cao có sẵn.
- **Bộ nhớ RAM:** 16GB Unified Memory.
- **Card đồ họa (GPU):** Apple M4 GPU (10 nhân).

Lưu ý: GPU chỉ đóng vai trò hiển thị, không tham gia vào quá trình tính toán lõi nhằm cô lập năng lực xử lý của CPU.

3.5.2 Cấu hình phần mềm và Môi trường thực thi

Các công cụ và nền tảng phần mềm được sử dụng bao gồm:

- **Hệ điều hành:** MacOS Tahoe.
- **Trình duyệt web:** Google Chrome phiên bản 113+. Đây là môi trường thực thi chính, hỗ trợ đầy đủ các đặc tả Wasm SIMD và SharedArrayBuffer.
- **Trình biên dịch:** Emscripten (emcc) phiên bản 5.0.6.
- **Môi trường Server:** Python-based HTTP Server được cấu hình để gửi các tiêu đề phản hồi (Response Headers) đặc thù:
 - Cross-Origin-Opener-Policy
 - Cross-Origin-Embedder-Policy

3.5.3 Tham số biên dịch và Dữ liệu đầu vào

Để đạt được hiệu suất tối ưu cho các mô-đun Wasm, các cờ hiệu biên dịch sau được áp dụng:

- **Mức tối ưu hóa:** -O3 (Mức cao nhất để tối ưu hóa thời gian thực thi).
- **SIMD:** -msimd128 (Kích hoạt tập lệnh 128-bit).
- **Multi-threaded:** -pthread kết hợp -s PTHREAD_POOL_SIZE=4 (Thiết lập sẵn 4 luồng để loại bỏ độ trễ khởi tạo).

Dữ liệu thử nghiệm: Hệ thống sử dụng tệp tin video mẫu định dạng MP4, độ phân giải 4K UHD (3840 x 2160 pixels). Việc sử dụng dữ liệu 4K giúp tạo ra tải tính toán đủ lớn để bộc lộ rõ rệt sự khác biệt về hiệu năng giữa các kịch bản.

3.6 Phương pháp Benchmark và Thu thập dữ liệu

Để các kết quả thực nghiệm có giá trị thuyết phục và khách quan, hệ thống triển khai một giao thức đo lường chuẩn hóa, tập trung vào việc cô lập hiệu năng thực thi lõi và xử lý dữ liệu theo các phương pháp thống kê hiện đại.

3.6.1 Các điểm đo lường

Hệ thống phân tách rõ rệt giữa "Thời gian xử lý thuật toán" và "Thời gian thực thi hệ thống". Các điểm chốt (checkpoints) được đặt tại các vị trí chiến lược trong mã nguồn để thu thập dữ liệu:

- **Latency xử lý trên mỗi khung hình:** Được đo bằng `API performance.now()`. Điểm bắt đầu được đặt ngay trước khi gọi hàm xử lý pixel và điểm kết thúc được đặt ngay sau khi hàm xử lý trả về. Cách đo này loại bỏ các chi phí nằm ngoài vòng xử lý như giải mã video và vẽ kết quả lên Canvas.
- **Đối với JavaScript,** latency bao gồm thời gian thực thi toàn bộ hàm xử lý ảnh bằng JavaScript, bao gồm chuyển ảnh xám, Box Blur, so sánh khung hình và tạo dữ liệu ảnh đầu ra.
- **Đối với WebAssembly,** latency được đo bao quanh toàn bộ lời gọi ``window.wasmMotion.process(...)``. Vì vậy, giá trị này bao gồm cả chi phí cấp phát vùng nhớ đầu vào, sao chép dữ liệu từ JavaScript sang Wasm Linear Memory, thực thi hàm C++, sao chép kết quả ngược về JavaScript và giải phóng vùng nhớ đầu vào. Trong cài đặt hiện tại, hệ thống chưa tách riêng một biến đo độc lập cho chi phí cầu nối Wasm. Do đó, latency của Wasm phản ánh chi phí xử lý thực tế khi tích hợp vào pipeline trình duyệt, thay vì chỉ là thời gian tính toán thuần trong C++.

3.6.2 Quy trình lấy mẫu và Giai đoạn khởi động

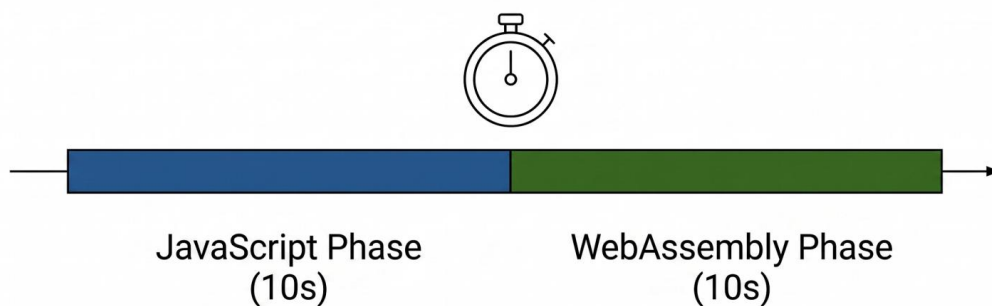
Để đảm bảo tính khách quan và loại bỏ các sai số tức thời, quy trình thực nghiệm được chia thành hai đợt đo lường liên tiếp trong cùng một phiên làm việc:

1. Giai đoạn đo lường JavaScript (10 giây đầu)

- **Khởi động ngầm:** Ngay khi bắt đầu, Engine V8 sẽ thực hiện biên dịch JIT và tối ưu hóa mã nguồn JavaScript.
- **Lấy mẫu:** Hệ thống ghi nhận hiệu năng của JavaScript (FPS, thời gian xử lý) liên tục trong 10 giây. Việc lấy mẫu trong khoảng thời gian này giúp bao quát được các kịch bản trình duyệt thực hiện việc quản lý bộ nhớ và các biến động CPU thông thường.

2. Giai đoạn đo lường WebAssembly (10 giây tiếp theo)

- **Chuyển đổi tức thì:** Ngay sau khi kết thúc 10 giây của JavaScript, hệ thống tự động kích hoạt module Wasm để xử lý cùng một luồng dữ liệu video 4K.
- **Đặc tính thực thi:** Khác với JavaScript, Wasm không cần tốn nhiều thời gian cho việc "soi" code, nên sẽ đạt trạng thái hiệu năng đỉnh gần như ngay lập tức.
- **Lấy mẫu:** Ghi nhận dữ liệu thực thi của Wasm trong 10 giây tiếp theo để so sánh trực tiếp với kết quả của JavaScript trên cùng một điều kiện môi trường và trạng thái phần cứng.



Hình 3.5 Hai giai đoạn của benchmark

3.6.3 Phương pháp xử lý thống kê

Dữ liệu thô sau khi thu thập được xử lý qua các chỉ số thống kê để đánh giá toàn diện hiệu năng:

- **Giá trị trung bình (Mean) và Trung vị (Median):** Đánh giá hiệu suất tổng thể của hệ thống. Trung vị được ưu tiên sử dụng để phản ánh hiệu năng thực tế vì nó ít bị ảnh hưởng bởi các giá trị ngoại lai.
- **Phân vị thứ 99 (P99 Latency):** Chỉ số này cực kỳ quan trọng trong xử lý video thời gian thực. Nó đại diện cho độ trễ trong tình huống xấu nhất, giúp đánh giá hiện tượng "giật hình".
- **Độ lệch chuẩn (Standard Deviation):** Dùng để đo lường tính ổn định của môi trường thực thi. Độ lệch chuẩn càng thấp chứng tỏ engine thực thi càng có tính xác định cao.
- **Thông lượng (Throughput/FPS):** Được đo từ khoảng thời gian giữa các khung hình xử lý liên tiếp trong vòng lặp *requestAnimationFrame*. Chỉ số này phản ánh thông lượng đầu cuối của pipeline trình duyệt, bao gồm cả ảnh hưởng của lịch trình render và các chi phí tích hợp ngoài thời gian xử lý lõi.

CHƯƠNG 4: KẾT QUẢ VÀ PHÂN TÍCH

4.1 Giới thiệu chương

Chương này trình bày và phân tích chi tiết các kết quả thu được từ quá trình thực nghiệm hệ thống benchmark đã thiết kế tại Chương 3. Mục tiêu cốt lõi là đưa ra các bằng chứng định lượng để so sánh hiệu năng giữa JavaScript và Wasm trong bài toán xử lý video độ phân giải 4K (3840 x 2160 pixels) thông qua bốn kịch bản tối ưu hóa tích lũy.

Để đảm bảo tính khách quan và phản ánh trung thực hành vi thực thi của hệ thống trong môi trường trình duyệt, nghiên cứu này sử dụng phương pháp lấy mẫu dựa trên thời gian thay vì giai đoạn khởi động truyền thống. Cụ thể, mỗi kịch bản thực nghiệm được thực thi liên tục trong một cửa sổ thời gian tổng cộng 20 giây cho mỗi lượt đo, phân tách rõ rệt thành hai phân đoạn: 10 giây đầu tiên thực thi thuần túy trên môi trường JavaScript và 10 giây tiếp theo chuyển thực thi sang Wasm. Cách tiếp cận này cho phép quan sát và phân tích chi tiết sự biến động của các chỉ số hiệu năng (Latency, FPS, P99, Standard Deviation) ngay từ thời điểm khởi tạo, qua đó đánh giá được tác động trực tiếp của quá trình tối ưu hóa JIT và áp lực quản lý bộ nhớ lên sự ổn định của hệ thống trong suốt thời gian thực thi thực tế.

Nội dung của chương được cấu trúc theo mạch phân tích từ cơ bản đến chuyên sâu, bao gồm:

- **Phân tích hiệu năng đơn luồng (Scalar):** Đối chiếu trực tiếp giữa engine thực thi của JavaScript và Wasm.
- **Đánh giá các tầng tối ưu hóa phần cứng:** Phân tích mức độ cải thiện khi áp dụng lần lượt tập lệnh SIMD 128-bit và mô hình đa luồng (Multi-threaded) qua pthreads.
- **Đánh giá kịch bản tích lũy (Hybrid):** Xác định giới hạn hiệu năng thực tế khi kết hợp đồng thời các kỹ thuật tối ưu hóa thuật toán và phần cứng.
- **Thảo luận và Biện giải:** Phân tích bản chất chuyển dịch từ giới hạn tính toán (Compute-bound) sang giới hạn băng thông bộ nhớ (Memory-bound) và so sánh kết quả thực tế với giới hạn lý thuyết của Định luật Amdahl.

Các kết quả này sẽ là cơ sở thực chứng quan trọng để đưa ra những kết luận về khả năng thay thế hoặc bổ trợ của Wasm đối với JavaScript trong các ứng dụng web hiệu năng cao hiện nay.

4.2 Kết quả thực nghiệm các kịch bản đơn lẻ

Trong phần này, báo cáo tập trung đánh giá hiệu năng của các kỹ thuật tối ưu hóa khi hoạt động độc lập nhằm xác định mức đóng góp của từng tầng công nghệ vào tốc độ xử lý tổng thể của hệ thống.

4.2.1 Kịch bản 1: JavaScript và WebAssembly Scalar

Kịch bản 1 thiết lập đường cơ sở (baseline) để so sánh hiệu năng giữa hai môi trường thực thi mà không có sự hỗ trợ của các kỹ thuật tối ưu hóa phần cứng đặc biệt. Mục tiêu là đo lường lợi ích thuần túy từ cơ chế biên dịch và quản lý bộ nhớ.

Bảng 4.1 Kịch bản 1: WebAssembly scalar

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	35	126	
FPS trung bình	3.4	11.4	+237.1%
Mean (ms)	273.74ms	61.22ms	-77.6%
Median (ms)	258.70ms	60.70ms	-76.5%
P99	462.80ms	64.10ms	-86.1%
Độ lệch chuẩn (σ)	40.82ms	2.89ms	-92.9%

Phân tích và Nhận xét:

- **Sự bứt phá về thông lượng:** Wasm đạt mức FPS trung bình là 11.4, cao gấp 3.35 lần so với JavaScript (3.4). Điều này khẳng định rằng ngay cả khi chưa áp dụng SIMD hay đa luồng, việc thực thi mã nhị phân kiểu tĩnh đã mang lại lợi thế vượt trội trong bài toán xử lý 8.3 triệu pixel mỗi khung hình.
- **Lợi thế từ cơ chế biên dịch AOT:** Độ trễ trung bình của Wasm giảm mạnh xuống còn 61.22 ms (so với 273.74 ms của JavaScript). Kết quả này minh chứng cho hiệu quả của cơ chế biên dịch Ahead-of-Time, giúp giảm chi phí suy luận kiểu dữ liệu trong đường thực thi nóng và hạn chế các rủi ro warm-up hoặc deoptimization thường gặp ở JavaScript JIT.
- **Tính ổn định hệ thống:** Chỉ số P99 và Độ lệch chuẩn σ cho thấy sự khác biệt mang tính quyết định.
 - Trong khi JavaScript có độ lệch chuẩn lên tới 40.82 ms, gây ra hiện tượng giật hình nặng nề, thì Wasm duy trì mức σ cực thấp là 2.89 ms (giảm 92.9%).
 - Giá trị P99 của JavaScript (462.80 ms) cao gần gấp đôi so với Median, phản ánh các "đỉnh" trễ do cơ chế quản lý bộ nhớ tự động bị kích hoạt để dọn dẹp các vùng nhớ đệm pixel bị cấp phát liên tục.
 - Ngược lại, Wasm thể hiện tính xác định cao nhờ chiến lược tái sử dụng vùng đệm và quản lý bộ nhớ thủ công trên Linear Memory.

4.2.2 Kịch bản 2: Hiệu quả của tối ưu hóa mức dữ liệu (SIMD)

Kịch bản này đánh giá tác động của việc vector hóa các phép tính trên pixel bằng tập lệnh SIMD 128-bit so với đường cơ sở JavaScript. Việc áp dụng SIMD cho phép hệ thống thực hiện song song hóa mức dữ liệu, một bước tiến quan trọng so với cơ chế thực thi tuần tự truyền thống.

Bảng 4.2 Kịch bản 2: WebAssembly SIMD

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	35	218	
FPS trung bình	3.3	20.7	+526.9%
Mean (ms)	284.47ms	23.72ms	-91.7%
Median (ms)	271.30ms	23.40ms	-91.4%
P99	454.50ms	25.90ms	-94.3%
Độ lệch chuẩn (σ)	42.20ms	1.44ms	-96.6%

Phân tích và Nhận xét:

- **Sự bứt phá về thông lượng tính toán:** Kịch bản SIMD đạt 20.7 FPS, cao hơn đáng kể so với JavaScript baseline. Mức cải thiện này đến từ tổ hợp nhiều tối ưu: chuyển grayscale sang số nguyên, thay Box Blur naive bằng separable blur kết hợp sliding window, và dùng SIMD 128-bit cho bước frame differencing/output.
- **Tối ưu hóa tập lệnh và giảm rẽ nhánh:** Ở bước diff/output, SIMD xử lý đồng thời 16 giá trị grayscale 8-bit, sử dụng mặt nạ bit để tạo kết quả và đếm pixel thay đổi. Tuy nhiên, cần lưu ý rằng bước grayscale trong code hiện tại vẫn là vòng lặp tuần tự, còn phần tăng tốc lớn của kịch bản này cũng đến từ thay đổi thuật toán blur.
- **Tính ổn định:** Độ lệch chuẩn σ giảm xuống 1.44 ms và P99 đạt 25.90 ms, cho thấy thời gian xử lý ổn định hơn JavaScript baseline trong thực nghiệm này.

4.2.3 Kịch bản 3: Hiệu quả của tối ưu hóa mức luồng (Multi-threaded)

Kịch bản này đo lường khả năng mở rộng hiệu năng khi phân phối khối lượng tính toán cực lớn của video 4K lên đa lõi CPU.

Bảng 4.3 Kịch bản 3: WebAssembly Multi-threaded

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	36	184	
FPS trung bình	3.4	17.3	+405.9%
Mean (ms)	273.9ms	33.75ms	-87.7%
Median (ms)	262.65ms	32.95ms	-87.5%
P99	487.25ms	39.71ms	-91.9%
Độ lệch chuẩn (σ)	44.26ms	7.75ms	-82.5%

Phân tích và Nhận xét:

- **Hiệu quả song song hóa:** Wasm đa luồng giúp tăng FPS lên 17.3, gấp hơn 5 lần so với JavaScript. Việc sử dụng SharedArrayBuffer cho phép các luồng truy cập trực tiếp vào cùng một vùng nhớ, loại bỏ chi phí sao chép dữ liệu khổng lồ giữa các Web Workers.
- **Giới hạn lý thuyết (Định luật Amdahl):** Mặc dù tốc độ tăng đáng kể, nhưng hiệu năng không tăng tuyến tính theo số luồng. Điều này do sự tồn tại của các phần mã thực thi tuần tự không thể song song hóa, như chi phí điều phối luồng và rào cản đồng bộ hóa.

Tại sao σ kịch bản này lại cao hơn kịch bản 1?

- **Chi phí điều phối (Scheduling Overhead):** Khi chạy đa luồng, hệ điều hành phải liên tục phân phối tài nguyên cho 4 luồng khác nhau. Sự biến động về thời gian cấp phát CPU của OS giữa các khung hình gây ra nhiễu về độ trễ, làm tăng σ .
- **Rào cản đồng bộ:** Hệ thống dùng mô hình Fork-Join, khung hình chỉ hoàn thành khi luồng cuối cùng xong việc. Một luồng bị trễ do tắc nghẽn sẽ kéo dài toàn bộ pipeline, tạo ra sự thiếu ổn định cao hơn đơn luồng.
- **Tranh chấp tài nguyên:** Các luồng phải chia sẻ băng thông bộ nhớ và bộ nhớ đệm khi truy xuất 33MB dữ liệu của video 4K. Xung đột truy cập ngẫu nhiên làm giảm tính xác định so với việc chỉ dùng 1 luồng duy nhất.

4.3 Kết quả kịch bản tích lũy (SIMD + Multi-threaded)

Kịch bản cuối cùng đánh giá sức mạnh tổng hợp khi kết hợp song song hóa mức dữ liệu và song song hóa mức luồng. Đây là mô hình tối ưu hóa cao nhất nhằm khai thác triệt để kiến trúc phần cứng hiện đại.

Bảng 4.4 Kịch bản 4: WebAssembly SIMD + Multi-threaded

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	36	290	
FPS trung bình	3.4	27.9	+711.6%
Mean (ms)	272.24ms	11.13ms	-95.9%
Median (ms)	256.05ms	11.12ms	-95.7%
P99	440.37ms	13.17ms	-97%
Độ lệch chuẩn (σ)	38.96ms	1.29ms	-96.7%

a. Phân tích và Nhận xét:

- **Đạt ngân sách tính toán của 60 FPS ở mức lỗi xử lý:** Với độ trễ 11.13 ms, lỗi xử lý thuật toán thấp hơn ngưỡng 16.67 ms tương ứng với mục tiêu 60 FPS. Tuy nhiên, đây chưa phải FPS đầu cuối của toàn hệ thống. Khi cộng thêm chi phí sao chép dữ liệu qua JavaScript-Wasm bridge và hiển thị lên Canvas, FPS trung bình đo được là 27.9 FPS.
- **Hiệu ứng cộng hưởng:** Sự kết hợp SIMD và Multi-threading tạo ra mức tăng tốc phi tuyến tính (gấp ~8 lần JavaScript). SIMD làm nhẹ tải tính toán trên mỗi lõi, giúp việc đồng bộ hóa đa luồng diễn ra nhịp nhàng và ổn định hơn.
- **Độ ổn định cao:** Chỉ số $\sigma = 1.29$ ms cho thấy thời gian xử lý lỗi của kịch bản Hybrid biến động rất thấp so với JavaScript baseline. Điều này chứng minh Wasm kết hợp SIMD và đa luồng giúp giảm đáng kể jitter ở tầng xử lý thuật toán, dù FPS đầu cuối vẫn còn phụ thuộc vào bridge và hiển thị.

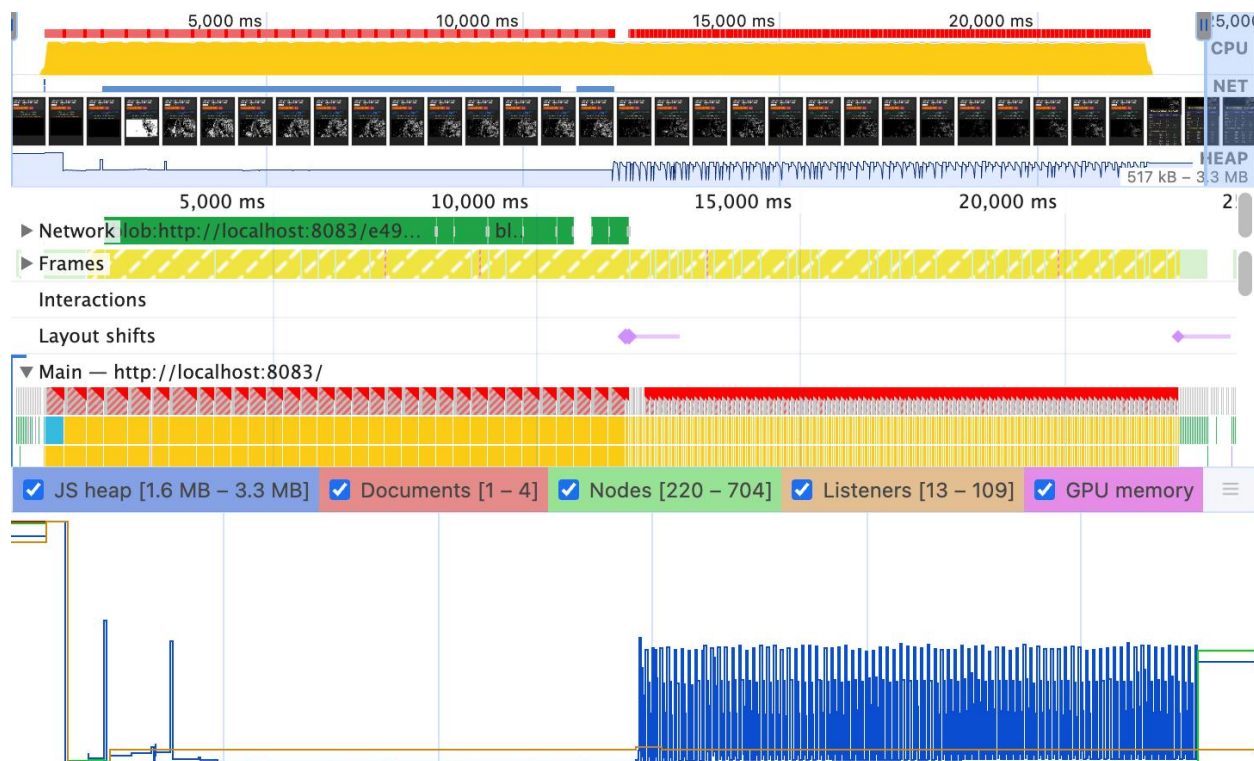
b. Ngưỡng bão hòa hiệu năng:

Kết quả thực nghiệm tại kịch bản này chỉ ra một quan sát quan trọng: **Hệ thống đã tiệm cận điểm bão hòa về tính toán.**

- **Sự chuyển dịch nút thắt cổ chai:** Khi độ trễ tính toán giảm xuống mức ~11ms, rào cản chính của hệ thống không còn là năng lực xử lý của CPU mà chuyển sang băng thông bộ nhớ. Chi phí truy cập và sao chép dữ liệu 4K mỗi khung hình,

đặc biệt giữa JavaScript Heap và Wasm Linear Memory, bắt đầu chiếm tỷ trọng lớn trong tổng thời gian thực thi.

- **Giới hạn vật lý:** Việc tiếp tục tăng số luồng hoặc mở rộng tập lệnh vector lúc này sẽ mang lại lợi ích biên giảm dần. Điều này khẳng định cấu hình Hybrid (SIMD + MT) đã khai thác gần như trọn vẹn giới hạn vật lý của trình duyệt trên kiến trúc phần cứng hiện tại.



Hình 4.1 Performance panel của chrome trong kịch bản số 4

Dù Wasm có bộ nhớ tuyến tính cố định và ổn định, nhưng trong kịch bản số 4, bộ nhớ Heap của trình duyệt vẫn biến thiên liên tục (hình 4.1) là do cơ chế hoạt động của lớp Cầu nối (Wasm Bridge) và cách trình duyệt hiển thị hình ảnh.

Chi phí Wasm bridge: Mỗi khi Wasm xử lý xong một khung hình, mã JavaScript phải lấy dữ liệu kết quả ra để hiển thị. Với video 4K, mỗi khung hình chiếm khoảng 33MB. Biểu đồ **JavaScript Heap** tăng rất nhanh và rơi thẳng đứng chứng tỏ bộ nhớ được cấp phát ồ ạt và bị dọn dẹp ngay lập tức. Quá trình cấp phát và thu gom bộ nhớ này tạo thêm áp lực lên main thread, làm hạn chế khả năng tăng FPS đầu cuối dù thời gian xử lý lỗi đã được tối ưu mạnh.

Kết luận: Dù Wasm kết hợp multi-thread với SIMD giúp tăng tốc độ xử lý nhưng đến ngưỡng cấu trúc, việc copy-transfer lại là giới hạn. Tại ngưỡng bão hòa, phần lợi ích từ tối ưu thuật toán giảm dần, trong khi chi phí cấp phát, sao chép và thu gom bộ nhớ ở tầng tích hợp bắt đầu chiếm tỷ trọng lớn hơn trong FPS đầu cuối.

4.4. Phân tích chuyên sâu và Thảo luận

Dựa trên các kết quả thực nghiệm từ 4 kịch bản tối ưu hóa tích lũy, nghiên cứu tiến hành phân tích sâu các yếu tố ảnh hưởng đến hiệu năng và xác định các giới hạn thực thi trong môi trường trình duyệt.

4.4.1. Hiệu quả cộng hưởng của tối ưu hóa tích lũy

Kết quả cho thấy một sự tăng trưởng hiệu năng phi tuyến tính khi kết hợp các kỹ thuật tối ưu hóa.

- **SIMD (KB2)** kết hợp tối ưu thuật toán blur, chuyển đổi số nguyên và SIMD ở bước diff/output để giảm chi phí xử lý trên một luồng.
- **Multi-threading (KB3)** tập trung vào việc phân phối tải tính toán.
- Khi tích hợp thành mô hình Hybrid (KB4), hệ thống đạt mức tăng tốc lên tới 711.6% so với Baseline. Sự cộng hưởng này xuất phát từ việc SIMD làm giảm đáng kể thời gian chiếm dụng CPU của mỗi luồng, từ đó rút ngắn "rủi ro" đối với các tác vụ điều phối của hệ điều hành, giúp việc đồng bộ hóa đa luồng diễn ra với độ trễ thấp kỷ lục ($\sigma = 1.29\text{ms}$).

4.4.2. Phân tích nút thắt cổ chai tại ngưỡng bão hòa hiệu năng

Mặc dù các kỹ thuật tối ưu hóa mã nguồn đã đạt tới mức cực hạn, hiệu năng hệ thống bắt đầu có dấu hiệu hội tụ và không thể tăng thêm đáng kể. Qua phân tích sâu bằng công cụ Performance Panel, nghiên cứu xác định nút thắt cổ chai đã chuyển dịch từ **Tính toán** sang **Quản lý Bộ nhớ**.

- **Hiện tượng Memory Churn:** Video 4K với lưu lượng ~33MB/khung hình tạo ra áp lực cấp phát bộ nhớ khổng lồ. Hình 4.1 cho thấy các dải thực thi bị ngắt quãng liên tục bởi các tác vụ quản lý bộ nhớ. Engine của trình duyệt phải tạm dừng luồng thực thi chính để dọn dẹp Heap, gây ra các đỉnh trễ (P99) và chặn đứng khả năng tăng FPS.
- **Giới hạn băng thông bộ nhớ:** Tại Kịch bản 4, thời gian CPU thực hiện thuật toán lỗi đã giảm xuống mức tối thiểu (~11ms). Lúc này, chi phí thời gian bắt đầu bị chi phối nhiều hơn bởi truy cập bộ nhớ lớn, copy-in/copy-out giữa JavaScript Heap và Wasm Linear Memory, cùng chi phí tạo dữ liệu hiển thị. Việc thêm tải nguyên tính toán như tăng số luồng không còn mang lại hiệu quả biên tương xứng do băng thông bộ nhớ đã đạt ngưỡng giới hạn.

4.4.3. Tính xác định và ổn định của WebAssembly

Một điểm khác biệt trọng yếu giữa JavaScript và Wasm là tính xác định.

- Trong JavaScript, engine JIT có thể thực hiện tái tối ưu hóa bất ngờ, kết hợp với cơ chế quản lý bộ nhớ tự động, dẫn đến độ lệch chuẩn cao ($\sigma \approx 40\text{ms}$).
- Wasm với lợi thế mã nhị phân kiểu tĩnh và khả năng quản lý bộ nhớ chủ động trong lõi C++ duy trì độ biến động thấp hơn đáng kể so với JavaScript trong thực

nghiệm này. Kết quả cho thấy Wasm là một hướng tiếp cận phù hợp cho các pipeline xử lý video cần độ trễ ổn định trên trình duyệt, đặc biệt khi kết hợp với SIMD và đa luồng.

4.5 Tổng kết chương

Chương 4 đã thực hiện phân tích thực nghiệm chi tiết và đa chiều về hiệu năng của Wasm so với JavaScript trong bài toán xử lý video 4K thời gian thực. Thông qua quá trình đo lường và biện giải, nghiên cứu đã rút ra được các kết luận then chốt sau:

1. **Sự vượt trội của WebAssembly:** Wasm Scalar cung cấp một mức nền tảng hiệu năng cao hơn và ổn định hơn đáng kể so với JavaScript. Việc loại bỏ hiện tượng jitter do cơ chế quản lý bộ nhớ tự động gây ra đã chứng minh Wasm là môi trường thực thi lý tưởng cho các tác vụ mang tính xác định cao.
2. **Hiệu quả tối ưu hóa phần cứng:** Kịch bản SIMD giúp giảm mạnh độ trễ nhờ kết hợp nhiều tầng tối ưu: chuyển grayscale sang số nguyên, thay Box Blur naive bằng separable blur kết hợp sliding window, và vector hóa bước frame differencing/output bằng SIMD 128-bit.
3. **Ngưỡng bão hòa hiệu năng:** Kết quả thực nghiệm tại kịch bản tích lũy cao nhất (KB4) đã chỉ ra sự chuyển dịch trạng thái của hệ thống từ giới hạn tính toán sang giới hạn băng thông bộ nhớ. Đây là một chỉ dấu quan trọng cho thấy hệ thống đã tiệm cận giới hạn vật lý của môi trường thực thi trên trình duyệt.
4. **Tính khả thi của ứng dụng:** Kết quả cho thấy Wasm là một hướng tiếp cận khả thi cho các tác vụ thị giác máy tính độ phân giải cao trên trình duyệt, đặc biệt khi kết hợp tối ưu thuật toán, SIMD và đa luồng. Tuy nhiên, hiệu năng đầu cuối vẫn phụ thuộc đáng kể vào chi phí bridge và hiển thị.

Những kết quả định lượng này không chỉ minh chứng cho các giả thuyết khoa học đã đặt ra mà còn là cơ sở thực chứng vững chắc để đưa ra các kết luận cuối cùng và đề xuất hướng phát triển ở chương tiếp theo.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

5.1.1 Đối chiếu kết quả thực nghiệm với câu hỏi nghiên cứu

Dựa trên các kết quả định lượng ở Chương 4, nghiên cứu có thể trả lời trực tiếp bốn câu hỏi nghiên cứu đã đặt ra ở **Mục 1.3.3** như sau:

RQ1: Trong môi trường xử lý video 4K với khối lượng dữ liệu pixel lớn, phiên bản Wasm Scalar mang lại hệ số tăng tốc như thế nào so với mã nguồn JavaScript truyền thống?

- **Trả lời:** Kết quả thực nghiệm cho thấy Wasm Scalar mang lại cải thiện hiệu năng rõ rệt so với JavaScript truyền thống trong môi trường xử lý video 4K:
 - Ở kịch bản Scalar, độ trễ trung bình của JavaScript là 273.74 ms/khung hình, trong khi Wasm Scalar chỉ còn 61.22 ms/khung hình.
 - Nếu xét theo độ trễ xử lý, Wasm Scalar đạt hệ số tăng tốc xấp xỉ 4.47 lần.
 - Nếu xét theo thông lượng đầu cuối, FPS trung bình tăng từ 3.4 FPS lên 11.4 FPS, tương ứng mức cải thiện khoảng 3.35 lần.
 - Kết quả này cho thấy ngay cả khi chưa áp dụng SIMD hay đa luồng, Wasm đã có lợi thế đáng kể nhờ cơ chế thực thi mã nhị phân kiểu tĩnh, biên dịch trước và mô hình bộ nhớ tuyến tính ổn định hơn so với JavaScript.

RQ2: Việc áp dụng tập lệnh SIMD giúp tối ưu hóa thời gian xử lý khung hình ở mức độ nào đối với các phép toán xử lý ảnh cơ bản?

- **Trả lời:** Việc áp dụng SIMD đem lại mức tối ưu hóa đáng kể cho các phép toán xử lý ảnh cơ bản trên từng điểm ảnh.
 - Khi sử dụng Wasm SIMD, độ trễ trung bình giảm xuống còn 23.72 ms/khung hình, thấp hơn 2.58 lần so với Wasm Scalar và thấp hơn khoảng 12 lần so với JavaScript trong cùng nhóm thực nghiệm.
 - FPS trung bình đạt 20.7 FPS, cho thấy SIMD giúp tăng đáng kể thông lượng xử lý pixel nhờ khả năng thực hiện song song nhiều phép toán dữ liệu trong một chu kỳ lệnh.
 - Ngoài tốc độ, SIMD còn cải thiện tính ổn định khi độ lệch chuẩn chỉ còn 1.44 ms và P99 đạt 25.90 ms, thấp hơn rất nhiều so với JavaScript.

RQ3: Khi triển khai đa luồng trong Wasm, hiệu năng thực tế có đạt được sự tăng trưởng tuyến tính hay bị giới hạn bởi chi phí quản lý luồng và băng thông bộ nhớ của trình duyệt?

- **Trả lời:** Kết quả cho thấy đa luồng trong Wasm giúp tăng hiệu năng thực tế nhưng không đạt tăng trưởng tuyến tính theo số luồng.
 - Ở kịch bản Wasm Multi-threaded, độ trễ trung bình đạt 33.75 ms/khung hình, tốt hơn đáng kể so với JavaScript 273.9 ms/khung hình và cũng cải thiện so với Wasm Scalar.
 - Tuy nhiên, khi so với Wasm Scalar, mức tăng tốc chỉ khoảng 1.81 lần, thấp hơn nhiều so với kỳ vọng tuyến tính khi sử dụng nhiều luồng. Nguyên nhân chính nằm ở chi phí điều phối luồng, đồng bộ hóa, tranh chấp bộ nhớ đệm và giới hạn băng thông bộ nhớ khi mỗi khung hình 4K có dung lượng xấp xỉ 33 MB.
 - Do đó, đa luồng là một hướng tối ưu hiệu quả, nhưng lợi ích của nó bị giới hạn bởi các phần xử lý tuần tự và chi phí quản lý tài nguyên trong trình duyệt.

RQ4: Kịch bản kết hợp SIMD và Multi-threaded có thực sự giúp kiểm soát độ trễ ổn định và ngăn chặn tình trạng mất đồng nhất (jitter) trong suốt quá trình xử lý video dài hay không?

- **Trả lời:** Kịch bản kết hợp SIMD và Multi-threaded là cấu hình hiệu quả nhất trong các thực nghiệm đã triển khai.
 - Kịch bản này đạt độ trễ xử lý lỗi trung bình 11.13 ms/khung hình, thấp hơn ngân sách 16.67 ms của mục tiêu 60 FPS. Tuy nhiên, FPS đầu cuối đo được là 27.9 FPS do còn chi phí bridge và hiển thị.
 - Đồng thời, P99 chỉ đạt 13.17 ms và độ lệch chuẩn là 1.29 ms, cho thấy hệ thống duy trì độ trễ ổn định và kiểm soát tốt hiện tượng jitter trong quá trình xử lý.
 - Như vậy, có thể kết luận rằng sự kết hợp giữa SIMD và đa luồng không chỉ cải thiện tốc độ trung bình mà còn nâng cao tính xác định của pipeline xử lý video.
 - Tuy nhiên, kết quả cũng chỉ ra rằng khi độ trễ tính toán đã giảm xuống khoảng 11 ms, nút thắt cổ chai chính không còn nằm ở thuật toán xử lý pixel mà chuyển sang chi phí di chuyển dữ liệu, băng thông bộ nhớ và cầu nối giữa JavaScript với Wasm.

Tổng hợp bốn câu hỏi nghiên cứu, có thể khẳng định rằng WebAssembly là một hướng tiếp cận hiệu quả cho xử lý video 4K thời gian thực trên trình duyệt, đặc biệt khi kết hợp các kỹ thuật tối ưu hóa tầng thấp như SIMD và đa luồng. Tuy nhiên, hiệu năng không tăng tuyến tính vô hạn. Khi các tầng tính toán đã được tối ưu, giới hạn thực thi

chuyển dần từ năng lực CPU sang băng thông bộ nhớ và chi phí sao chép dữ liệu trong kiến trúc trình duyệt.

Bảng 5.1 Tổng hợp câu trả lời cho các câu hỏi nghiên cứu

Câu hỏi	Nội dung trọng tâm	Kết quả chính
RQ1	So sánh Wasm Scalar vs. JavaScript	Wasm nhanh hơn 4.47 lần về độ trễ và 3.35 lần về chỉ số FPS.
RQ2	Tác động của SIMD	Giảm độ trễ xuống còn 23.72 ms/khung hình , nhanh hơn Wasm Scalar 2.58 lần .
RQ3	Hiệu quả của Multi-threaded	Hiệu năng cải thiện nhưng không tăng tuyến tính do rào cản từ đồng bộ luồng và băng thông bộ nhớ.
RQ4	Kết hợp SIMD + Multi-threaded	Đạt độ trễ lỗi 11.13 ms (P99: 13.17 ms, $\sigma = 1.29$ ms), thấp hơn ngân sách tính toán 60 FPS; FPS đầu cuối vẫn đạt 27.9 FPS .

5.1.2 Kết luận tổng hợp

Dựa trên các kết quả thực nghiệm hệ thống tại Chương 4, báo cáo rút ra các kết luận cốt lõi sau:

- **Vượt qua rào cản hiệu năng của JavaScript:** Wasm không chỉ đơn thuần là nhanh hơn, mà còn cung cấp một môi trường thực thi có tính xác định cao. Trong khi JavaScript gây ra hiện tượng jitter nặng nề do cơ chế quản lý bộ nhớ tự động, Wasm duy trì độ biến động thấp hơn đáng kể trong các kịch bản thực nghiệm, thể hiện qua độ lệch chuẩn thấp hơn JavaScript baseline.
- **Hiệu quả vượt trội của tối ưu hóa tích lũy:** Nghiên cứu đã chứng minh rằng sự kết hợp giữa SIMD 128-bit và Đa luồng tạo ra hiệu ứng cộng hưởng phi tuyến tính. Kịch bản này giúp thời gian xử lý lỗi đạt 11.13 ms/khung hình, tức thấp hơn ngưỡng 16.67 ms cần thiết cho ngân sách tính toán của 60 FPS. Tuy vậy, FPS đầu cuối của hệ thống vẫn chỉ đạt 27.9 FPS do pipeline trình duyệt còn bao gồm chi phí copy-in/copy-out, tạo ImageData và vẽ kết quả lên Canvas. Vì vậy, kết quả nên được hiểu là lỗi thuật toán đã đạt ngân sách 60 FPS, còn hệ thống end-to-end vẫn bị giới hạn bởi chi phí tích hợp.
- **Xác định điểm tới hạn vật lý:** Một đóng góp quan trọng của báo cáo là chỉ ra sự chuyển dịch nút thắt cổ chai. Khi tính toán đã được tối ưu đến mức cực hạn ($\sim 11\text{ms}$), rào cản chính không còn là CPU mà là băng thông bộ nhớ và chi phí sao chép dữ liệu qua "Cầu nối".

5.2 Đóng góp của báo cáo

Thông qua quá trình xây dựng hệ thống benchmark, triển khai các kịch bản tối ưu hóa và phân tích kết quả thực nghiệm, báo cáo đạt được một số đóng góp chính sau.

5.2.1 Về khoa học

1. Xác lập bộ dữ liệu thực nghiệm định lượng trên pipeline video 4K thực tế:

Nghiên cứu cung cấp hệ thống số liệu thực nghiệm về hiệu năng WebAssembly trong môi trường xử lý luồng dữ liệu liên tục cường độ cao (~33MB/khung hình). Khác với các nghiên cứu tập trung đánh giá vi mô (micro-benchmark), kết quả này phản ánh chính xác giới hạn của CPU, bộ nhớ và chi phí giao tiếp dữ liệu trong bối cảnh ứng dụng thực tế trên trình duyệt.

2. Phân tích tác động của tối ưu hóa tích lũy:

Thông qua việc thực nghiệm 4 kịch bản (Scalar, SIMD, Multi-threaded và Hybrid), nghiên cứu chứng minh hiệu năng hệ thống không tăng tuyến tính theo số lượng kỹ thuật áp dụng. Đóng góp quan trọng nằm ở việc nhận diện sự dịch chuyển nút thắt cổ chai từ năng lực tính toán sang băng thông bộ nhớ và chi phí quản lý vùng nhớ dùng chung.

3. Chuẩn hóa khung đánh giá bằng chỉ số ổn định:

Nghiên cứu khẳng định tầm quan trọng của các chỉ số P99 Latency và độ lệch chuẩn σ trong việc đánh giá trải nghiệm người dùng thực tế. Việc chứng minh giá trị trung bình (Mean/FPS) không đủ để phản ánh các hiện tượng giật hình đã thiết lập một khung tham chiếu tin cậy hơn cho các nghiên cứu về xử lý đa phương tiện thời gian thực trên Web.

5.2.2 Đóng góp về mặt kỹ thuật

1. Thiết kế khung thực nghiệm kiểm soát biến số:

Đồ án đã xây dựng hệ thống đo lường hiệu năng có khả năng cô lập các yếu tố nhiễu hệ thống (mạng, I/O, DOM). Việc triển khai các kịch bản thực nghiệm trên các ngữ cảnh thực thi độc lập giúp triệt tiêu ảnh hưởng của bộ nhớ đệm và trạng thái thực thi kế thừa, đảm bảo tính khách quan và độ tin cậy cao của dữ liệu số.

2. Nhất quán hóa thực nghiệm trên cùng một pipeline chức năng:

Hệ thống hiện thực hóa bài toán phát hiện chuyển động trên 5 cấu hình thực thi gồm JavaScript và 4 biến thể Wasm. Các cấu hình đều tuân theo cùng một pipeline chức năng: chuyển ảnh xám, làm mờ và so sánh khung hình. Tuy nhiên, các kịch bản tối ưu cao hơn có thay đổi cách cài đặt ở từng bước, chẳng hạn dùng số nguyên thay cho số thực, dùng separable blur thay cho blur 2D naive, và dùng SIMD hoặc đa luồng cho các đoạn phù hợp. Vì vậy, kết quả phản ánh hiệu quả của từng cấu hình tối ưu hóa hoàn chỉnh, thay vì chỉ cô lập duy nhất khác biệt về môi trường thực thi.

3. Nhận diện điểm tới hạn kiến trúc và định hướng tối ưu hóa hệ thống:

Nghiên cứu đã xác định được điểm nghẽn cổ chai của kiến trúc trình duyệt hiện tại: khi thời gian xử lý lỗi được tối ưu xuống mức cực hạn (~11ms/frame), chi phí sao chép dữ liệu giữa JavaScript Heap và Wasm Linear Memory trở thành rào cản chính. Đây là

phát hiện quan trọng, làm cơ sở kỹ thuật để đề xuất các hướng tối ưu hóa thể hệ mới như Zero-copy memory và tích hợp WebGPU.

5.2.3 Đóng góp về mặt thực tiễn

1. Xác lập chiến lược tối ưu hóa cho ứng dụng Web Media hiệu năng cao:

Kết quả nghiên cứu cung cấp lộ trình thực thi rõ ràng cho các nhà phát triển: khẳng định Wasm Scalar là bước nâng cấp thiết yếu từ JavaScript, trong khi SIMD là kỹ thuật then chốt để đạt được sự ổn định về độ trễ. Đồng thời, nghiên cứu cảnh báo việc áp dụng Đa luồng cần cân nhắc kỹ chi phí đồng bộ và giới hạn băng thông bộ nhớ thay vì kỳ vọng tăng tốc tuyến tính.

2. Chứng minh khả năng đưa lỗi xử lý video 4K về dưới ngân sách 60 FPS:

Bằng chứng thực nghiệm với cấu hình Hybrid (SIMD + Multi-threaded) đạt độ trễ xử lý lỗi khoảng 11.13 ms/khung hình, thấp hơn ngưỡng 16.67 ms của mục tiêu 60 FPS. Kết quả này cho thấy lỗi thuật toán phát hiện chuyển động có thể được tối ưu đủ nhanh trên trình duyệt hiện đại. Tuy nhiên, thông lượng đầu cuối vẫn bị giới hạn bởi chi phí trao đổi dữ liệu giữa JavaScript và Wasm cũng như chi phí hiển thị, nên cần tiếp tục tối ưu tầng bridge và bộ nhớ để đạt 60 FPS thực sự ở mức toàn pipeline.

3. Đề xuất bộ khuyến nghị kỹ thuật trong phát triển hệ thống:

Báo cáo cung cấp bộ quy tắc lựa chọn công nghệ dựa trên đặc thù tác vụ:

- Wasm Scalar: Dành cho chuyển đổi logic tuần tự từ JS.
- SIMD: Ưu tiên cho các toán tử pixel song song dữ liệu, đặc biệt khi có thể kết hợp vector hóa với tối ưu thuật toán như separable blur hoặc sliding window.
- Multi-threading: Chỉ áp dụng cho công việc có khối lượng cực lớn (4K+) sau khi đánh giá chi phí overhead.
- Hướng tối ưu cuối: Chuyển trọng tâm từ tăng luồng sang tối ưu hóa quản lý bộ nhớ.

5.3 Hạn chế và Hướng phát triển tương lai

5.3.1 Hạn chế của nghiên cứu

Mặc dù đạt được các kết quả định lượng khả quan về hiệu năng xử lý video 4K, nghiên cứu vẫn tồn tại một số giới hạn khách quan cần được xem xét:

1. Sự đa dạng của môi trường thực thi:

Kết quả nghiên cứu hiện tại chủ yếu dựa trên engine V8 (Google Chrome) và kiến trúc phần cứng Apple M4. Do sự khác biệt trong chiến lược biên dịch JIT, tối ưu hóa SIMD và quản lý bộ nhớ của các engine khác như SpiderMonkey (Firefox) hay JavaScriptCore (Safari), các chỉ số hiệu năng có thể biến thiên và chưa đại diện cho toàn bộ hệ sinh thái trình duyệt hiện nay.

2. Tính phổ quát của lớp bài toán:

Thực nghiệm mới tập trung vào pipeline phát hiện chuyển động – một bài toán xử lý pixel cường độ cao nhưng có cấu trúc tính toán tương đối đồng nhất. Kết quả này có thể chưa phản ánh chính xác hiệu năng của các lớp bài toán thị giác máy tính phức tạp hơn như nhận diện đối tượng đa mục tiêu hoặc suy luận mô hình Deep Learning vốn có đặc điểm truy cập bộ nhớ phi tuần tự.

3. Điểm nghẽn về kiến trúc bộ nhớ:

Dù kịch bản tối ưu Hybrid (SIMD + Multi-threading) đã đưa độ trễ xử lý lỗi xuống dưới ngân sách 60 FPS, hệ thống end-to-end vẫn chịu áp lực lớn từ chi phí sao chép dữ liệu (Copy-in/Copy-out) giữa JavaScript Heap và Wasm Linear Memory. Với dữ liệu 4K (~33MB/khung hình), rào cản về băng thông bộ nhớ và chi phí tích hợp trong môi trường trình duyệt là giới hạn mà tối ưu hóa thuật toán đơn thuần chưa thể giải quyết triệt để.

4. Phạm vi các chỉ số vận hành:

Nghiên cứu tập trung ưu tiên vào các chỉ số hiệu năng thời gian thực (Latency, FPS, P99). Các yếu tố quan trọng trong triển khai thực tế như mức tiêu thụ điện năng, nhiệt độ thiết bị và sự ảnh hưởng đến hiệu năng của các tiến trình web chạy song song chưa được đánh giá sâu trong phạm vi của báo cáo này.

5.3.2 Hướng phát triển tương lai

Dựa trên những kết quả đạt được và các hạn chế đã nhận diện, nghiên cứu có thể được mở rộng theo các hướng chủ đạo sau:

1. Mở rộng ma trận thực nghiệm đa nền tảng:

Triển khai đánh giá hiệu năng trên hệ sinh thái trình duyệt đa dạng (Firefox, Safari) và các kiến trúc phần cứng khác nhau (x86-64, thiết bị di động). Việc bổ sung các chỉ số về mức tiêu thụ năng lượng, nhiệt độ và hiệu dụng tài nguyên sẽ cung cấp bức tranh toàn diện về chi phí vận hành thực tế của WebAssembly.

2. Tối ưu hóa cơ chế trao đổi dữ liệu và bộ nhớ:

Nghiên cứu các kỹ thuật giảm thiểu chi phí sao chép dữ liệu (copy-in/copy-out) thông qua việc sử dụng bộ đệm vòng (ring buffer), tái sử dụng vùng nhớ hoặc ứng dụng các API hiện đại như WebCodecs. Mục tiêu trọng tâm là hướng tới mô hình zero-copy để giải quyết triệt để điểm nghẽn băng thông giữa JavaScript và Wasm Linear Memory.

3. Kiến trúc lai kết hợp WebAssembly và WebGPU:

Khảo sát mô hình phối hợp CPU-GPU, trong đó WebAssembly đảm nhiệm điều phối logic và tiền xử lý, còn WebGPU thực thi các kernel tính toán song song dữ liệu lớn (như tích chập, biến đổi ảnh). Đây là hướng tiếp cận tối ưu cho các pipeline thị giác máy tính phức tạp đòi hỏi năng lực xử lý vượt trội hơn so với chỉ dùng CPU.

4. Đa dạng hóa các lớp bài toán thị giác máy tính:

Mở rộng thực nghiệm sang các thuật toán nâng cao như Optical Flow, Segmentation hoặc nhận diện đối tượng theo thời gian thực. Việc này giúp xác định chính xác đặc thù của từng lớp bài toán để đưa ra chiến lược tối ưu phù hợp (SIMD, đa luồng hoặc xử lý lai) thay vì áp dụng một công thức chung.

5. Phát triển cơ chế điều phối tài nguyên thích nghi:

Xây dựng hệ thống tự động điều chỉnh chiến lược thực thi (số lượng luồng, cấu hình SIMD) dựa trên tải hệ thống thực tế và năng lực phần cứng của thiết bị đầu cuối. Cách tiếp cận này đảm bảo trải nghiệm người dùng mượt mà trên nhiều phân khúc thiết bị khác nhau, từ máy tính hiệu năng cao đến thiết bị di động giới hạn tài nguyên.

Tài liệu tham khảo

- [1] Jangda, A., Powers, B., Berger, E.D. and Guha, A. (2019) ‘Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code’, Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19), pp. 107–120. Available at: <https://www.usenix.org/conference/atc19/presentation/jangda>
- [2] Yan, Y., Tu, T., Zhao, L., Zhou, Y. and Wang, W. (2021) ‘Understanding the Performance of WebAssembly Applications’, Proceedings of the 21st ACM Internet Measurement Conference (IMC '21), pp. 533–549. doi: <https://dl.acm.org/doi/10.1145/3487552.3487827>
- [3] Taheri, S., Veidenbaum, A., Nicolau, A., Hu, N. and Haghighat, M.R. (2018) ‘OpenCV.js: Computer Vision Processing for the Open Web’, Proceedings of the 9th ACM Multimedia Systems Conference (MMSys '18), pp. 478–483. doi: <https://doi.org/10.1145/3204949.3208126>
- [4] Țălu, M. (2025) ‘A Comparative Study of WebAssembly Runtimes: Performance Metrics, Integration Challenges, Application Domains, and Security Features’, Archives of Advanced Engineering Science, pp. 1–13. doi: <https://doi.org/10.47852/bonviewAAES52024965>
- [5] Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A. and Bastien, J.F. (2017) ‘Bringing the Web up to Speed with WebAssembly’, Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '17), pp. 185–200. doi: <https://doi.org/10.1145/3140587.3062363>
- [6] Ciszewski, J. and Myk, K. (2025) ‘Performance Comparison of Web Assembly and JavaScript’, Journal of Pioneering Artificial Intelligence Research, 1(2), pp. 1–7. doi: <https://doi.org/10.63721/25JPAIR0111>
- [7] Oishi, S., Ishikawa, K., Nogami, H. and Fukushima, N. (2023) ‘Performance Evaluation of Image Convolution with WebAssembly’, Proceedings of SPIE: International Workshop on Advanced Imaging Technology (IWAIT) 2023, 12592, 125922G. doi: <https://doi.org/10.1117/12.2667004>
- [8] Vemuri, V.P.K. (2021) ‘WebAssembly for High-Performance Web Applications: A Study on Execution Speed and Efficiency’, International Journal on Science and Technology, 12(2). doi: <https://doi.org/10.71097/IJSAT.v12.i2.2812>
- [9] Parab, A. (2025) ‘WebAssembly’s Expanding Role in Frontend Development: Transforming Web Application Capabilities’, International Journal of Computational and Experimental Science and Engineering, 11(4), pp. 8694–8702. doi: <https://doi.org/10.22399/ijcesen.4281>